

ENHANCING STOCK MARKET INSIGHTS THROUGH HYBRID AI MODEL DRIVEN TECHNICAL ANALYSIS

*Project report submitted to
Visvesvaraya National Institute of Technology, Nagpur
in fulfillment of requirement for the award of
degree of*

Master of Technology
in
Applied Artificial Intelligence

by
Adwait Shirish Sonawane (MT23AAI031)

under the guidance of
Dr. Ankit Bhurane



Department of Applied Artificial Intelligence
Visvesvaraya National Institute of Technology, Nagpur
Nagpur 440010 (India)

December, 2025

ENHANCING STOCK MARKET INSIGHTS THROUGH HYBRID AI MODEL DRIVEN TECHNICAL ANALYSIS

*Project report submitted to
Visvesvaraya National Institute of Technology, Nagpur
in fulfillment of requirement for the award of
degree of*

Master of Technology
in
Applied Artificial Intelligence

by
Adwait Shirish Sonawane (MT23AAI031)

under the guidance of
Dr. Ankit Bhurane



Department of Applied Artificial Intelligence
Visvesvaraya National Institute of Technology, Nagpur
Nagpur 440010 (India)

December, 2025

Copyright © Visvesvaraya National Institute of Technology, Nagpur(VNIT)
December, 2025



DECLARATION

I, "Adwait Shirish Sonawane" hereby declare that the project work titled "**Enhancing Stock Market Insights Through Hybrid AI Model Driven Technical Analysis**" is carried out by me in the Applied Artificial Intelligence of VNIT, Nagpur, India. The work is original and has not been submitted earlier as a whole or in part for the award of any degree/diploma at this or any other Institution/University.

Adwait Shirish Sonawane
MT23AAI031

Date: December 01, 2025

CERTIFICATE

This is to certify that the thesis titled "**Enhancing Stock Market Insights Through Hybrid AI Model Driven Technical Analysis**" submitted by **Adwait Shirish Sonawane** in partial fulfilment of requirements for the award of Master of Technology in the Applied Artificial Intelligence of VNIT, Nagpur. The work is comprehensive, complete and fit for evaluation.

Dr. Ankit Bhurane
Assistant Professor,
Applied Artificial Intelligence, VNIT, Nagpur

Dr. Ashwin Kothari
Professor and Head,
Applied Artificial Intelligence, VNIT, Nagpur
Date: December 01, 2025

ACKNOWLEDGEMENTS

My sincere gratitude to my guide, for his constant support and encouragement, without which the project would not have been possible. My heartfelt gratitude to him for offering all of the facilities and assisting me in overcoming obstacles.

I would like to thank all the professors of ECE department department for helping me by some way or the other. I sincerely thank our project evaluators for their valuable comments and suggestions. I thank Head of ECE department for giving an opportunity for the project.

Date: December 01, 2025

Adwait Shirish Sonawane(MT23AAI031)

Abstract

Accurately forecasting short-term stock price movement remains a challenging problem due to the nonlinear, volatile, and dynamic nature of financial markets. Traditional forecasting approaches such as statistical models and manually applied technical analysis often struggle to adapt to changing market behavior and cannot fully capture relationships across time. With advancements in artificial intelligence, machine learning techniques now offer the potential to learn complex temporal patterns directly from historical data and improve predictive performance. This study applies machine learning and deep learning models to predict next-day closing prices for Indian stock market equities using technical analysis features.

Two years of daily historical stock data were collected through the Kite Connect API for companies listed in the NIFTY50, NIFTY200, and broader NSE universe. Each dataset includes OHLCV (Open, High, Low, Close, Volume) values and selected technical indicators such as Relative Strength Index (RSI), On-Balance Volume (OBV), Exponential Moving Average (EMA), and Average True Range (ATR). A sliding 20-day window is used as model input to predict the following day's closing value. The models implemented include LSTM, GRU, a hybrid CNN-LSTM, and a Transformer-based architecture. These models were trained and evaluated using a time-series split to preserve chronological integrity.

Performance was assessed using regression metrics including Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). A deployment interface was also developed to retrieve live data, generate predictions, and visualize model output over recent chart history, providing a practical layer for user interaction and lightweight backtesting.

Findings from the study indicate that deep learning architectures, particularly the CNN-LSTM and Transformer models, deliver improved prediction accuracy compared to single-sequence models, especially when trained on diverse, multi-stock datasets. Stock-specific LSTM and GRU models also performed well when aligned with individual price behavior. Overall, the research demonstrates that combining technical indicators with contemporary machine learning architectures can meaningfully enhance short-term forecasting performance and provide a foundation for future automated trading applications.

List of Figures

3.1	System Flow	20
3.2	Architecture of Data Flow.	30
5.1	Architecture of the GRU block.	35
5.2	Architecture of the LSTM block.	41
5.3	Architecture of the Transformer block.	47
5.4	Architecture of the CNN + LSTM block.	53
5.5	Conceptual Diagram of a CNN-LSTM Model Architecture	55
6.1	GRU without Indicators	58
6.2	GRU with Indicators Validation Behaviour	58
6.3	LSTM without Indicators Validation Behaviour	59
6.4	LSTM with Indicators Validation Behaviour	59
6.5	Transformers without Indicators Validation Behaviour	60
6.6	Transformers with Indicators Validation Behaviour	60
6.7	Regression Models on Single Company Data	61
6.8	Regression Models on Single Company Data With Indicators	62
6.9	Regression Models Compared Without Indicators Included	62
6.10	Regression Models Compared with Indicators Included	63
6.11	Prediction Errors for Regression Models on Single Company Data	63
6.12	GRU with stock data	65
6.13	GRU with Indicators included (EMA (12, 26), RSI (14), ATR (14), OBV)	65
6.14	LSTM with stock data	66
6.15	LSTM with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)	66
6.16	Transformers with stock data	67
6.17	Transformers with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)	67
6.18	CNN-LSTM with stock data	68
6.19	CNN-LSTM with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)	68

6.20	CNN-LSTM with Indicators (Close, MA5, MA20)	69
6.21	CNN-LSTM with Indicators (Close, EMA10, STD5)	69
6.22	CNN-LSTM with Indicators (Close, Return, STD20)	70
6.23	CNN-LSTM with Indicators Validation Loss Comparisons	70

List of Tables

2.1	Summary of Existing Machine Learning, Deep Learning, and Hybrid Approaches in Stock Market Forecasting	14
2.2	Comparison of machine learning, hybrid deep learning, and emerging AI approaches for stock market prediction.	16
3.1	Dataset Categorization Based on Usage and Scope	21
4.1	Recommended Hardware Specifications	31
4.2	Project Technology Stack	32
4.3	Dataset Splitting Strategy	33
4.4	Model Hyperparameters	33
4.5	Model Evaluation Metrics	34
5.1	Key Notations for Gated Recurrent Unit (GRU) Equations	37
5.2	Assessment of Model Characteristics (likely GRU/Simple RNN variant)	38
5.3	Model Architecture Summary	39
5.4	Summary of Technical Indicators	39
5.5	Stacked GRU Model Architecture Summary	39
5.6	Key notations and equations for the Long Short-Term Memory (LSTM) network	43
5.7	Assessment of LSTM Model Characteristics	44
5.8	Stacked LSTM Model Architecture Summary	45
5.9	Comparison of GRU vs. LSTM Architectures	45
5.10	Stacked LSTM Model Architecture Summary	45
5.11	Key notations and equations for the Transformer encoder layer used in this project	50
5.12	Analysis of a Deep Learning Model (Likely Transformer/Self-Attention-based) for Time Series Prediction	50
5.13	Model Architecture Summary	51

5.14	Model Architecture Summary	52
5.15	Comparison of Recurrent Neural Network (RNN) Architectures . . .	52
5.16	Analysis of a Hybrid Deep Learning Model (Likely CNN-LSTM) . . .	54
5.17	Model Architecture Summary	55
5.18	Model Architecture Summary (Likely CNN-LSTM)	56
6.1	Comparative Model Performance Across Datasets	64
6.2	Feature Set Performance Comparison	69
6.3	Strengths and Measures of Technical Indicator Sets	71
6.4	Summary of Technical Indicator Sets	71

Contents

Abstract	iii
List of Figures	iv
List of Tables	vi
1 Introduction	1
1.1 Introduction and Background	1
1.2 Evolution of Technical Analysis	2
1.3 Limitations of Classical Forecasting Models	2
1.4 Machine Learning as a Solution	3
1.5 Importance of Technical Indicators	3
1.6 Need for Hybrid Intelligent Systems	4
1.7 Identified Research Gap	4
1.8 Problem Statement	5
1.9 Research Objectives	5
1.10 Research Questions	6
1.11 Scope, Limitations, and Assumptions	6
1.12 Novelty and Contributions	7
2 Literature Review	8
2.1 Foundations of Stock Market Forecasting and Technical Analysis . . .	8
2.1.1 Introduction	8
2.1.2 Classical Forecasting Techniques	9
2.1.3 Technical Indicators and Traditional Trading Strategies	9
2.1.4 Traditional Machine Learning Models in Stock Forecasting . .	10
2.1.5 Summary	10
2.2 Emerging Trends in AI-Driven Stock Forecasting	11
2.3 Deep Learning Approaches	12

2.4	Hybrid Models Combining Technical Indicators and ML	13
2.5	Reinforcement Learning Approaches	13
2.6	Comparison Between different Model and research	13
2.7	Key Findings, Strengths, and Limitations	16
2.8	Evaluation Challenges, Research Gaps and Inconsistencies	18
2.9	Alignment with the Present Research	18
3	Methodology	20
3.1	Overview	20
3.2	System Workflow	20
3.3	Data Acquisition	21
3.4	Data Preprocessing	21
3.4.1	Handling Missing Values	21
3.4.2	Normalization	23
3.5	Feature Engineering	23
3.5.1	Raw Feature Selection	24
3.5.2	Temporal Windowing	24
3.5.3	Lagged Variables	24
3.5.4	Scaling and Normalization	25
3.5.5	Train-Test Structuring	25
3.5.6	Summary of Feature Engineering Process	25
3.6	Technical Indicators	26
3.6.1	Exponential Moving Average (EMA)	26
3.6.2	Relative Strength Index (RSI)	27
3.6.3	Average True Range (ATR)	28
3.6.4	On-Balance Volume (OBV)	28
3.6.5	Summary of Indicator Contribution	29
3.7	Windowing Strategy	29
3.8	Model Training and Validation	29
3.9	Prediction and Deployment Pipeline	30
4	Experimental Setup	31
4.1	Overview	31
4.2	Hardware Configuration	31
4.3	Software Stack	32
4.4	Dataset Preparation	32
4.5	Train-Test Split Strategy	32

4.6	Hyperparameter Configuration	33
4.7	Model-Specific Training Setup	33
4.7.1	LSTM And GRU Models	33
4.7.2	CNN-LSTM Model	33
4.7.3	Transformer Model	34
4.8	Evaluation Metrics	34
4.9	Deployment Testing	34
5	Model Details	35
5.1	Gated Recurrent Unit (GRU) Models	35
5.1.1	Introduction to GRU Architecture	35
5.1.2	Why GRUs Are Used in Time-Series Forecasting	36
5.1.3	Mathematical Model for GRU	36
5.1.4	Advantages and Disadvantages of GRUs Category Details . . .	37
5.1.5	Role of GRU in This Project	37
5.1.6	Trained GRU Model Configurations	37
5.1.7	Summary of GRU Model Performance	40
5.2	Long Short-Term Memory (LSTM) Model	41
5.2.1	Introduction to LSTM Architecture	41
5.2.2	Why LSTM Is Used for Time-Series Forecasting	42
5.2.3	Mathematical Model for LSTM	42
5.2.4	Advantages and Disadvantages of LSTM	43
5.2.5	Role of LSTM in This Project	43
5.2.6	Trained LSTM Model Configurations	44
5.2.7	Performance Summary	46
5.3	Transformer Model	47
5.3.1	Introduction to the Transformer Architecture	47
5.3.2	Motivation for Using Transformers in Time-Series Forecasting	47
5.3.3	Mathematical Model for Transformer Encoder	48
5.3.4	Advantages and Disadvantages of Transformers	50
5.3.5	Role of Transformers in This Project	50
5.3.6	Trained Transformer Model Configurations Transformer Model Without Indicators	51
5.3.7	Performance Summary	51
5.4	Conv1D + LSTM Hybrid Model	53
5.4.1	Introduction to the Hybrid Architecture	53
5.4.2	Motivation for Using a Hybrid Model in Time-Series Forecasting	53

5.4.3	Advantages and Disadvantages of the Hybrid Model	54
5.4.4	Role of the Hybrid Model in This Project	54
5.4.5	Trained Configurations of the Hybrid Model	55
5.4.6	Performance Summary	56
6	Results and Analysis	57
6.1	Overview	57
6.2	Training and Validation Behavior	57
6.3	Single Company Data Used	61
6.3.1	Quantitative Evaluation Results	64
6.4	Effect of Feature Engineering vs. Raw OHLCV Inputs	64
6.5	CNN-LSTM Feature Set Performance	69
6.5.1	Indicators tested and their purpose	71
6.6	Generalization vs. Specialization Behavior	72
6.7	Deployment and Practical Evaluation	73
7	Summary and Conclusions	74
7.1	Summary of Work and Key Contributions	74
7.2	Problem Statement and Solution	74
7.3	Major Findings and Significance	75
7.4	Limitations of the Study	76
7.5	Future Work	76
7.6	Conclusion	77

Chapter 1

Introduction

1.1 Introduction and Background

Financial markets play a critical role in the global economy by enabling capital allocation, investment, and wealth creation. The stock market, in particular, attracts researchers, practitioners, institutional investors, and retail traders due to its potential for returns and its inherently dynamic behavior. Forecasting stock price movements has long been considered a difficult and complex challenge because price fluctuations are influenced by numerous factors including market sentiment, macroeconomic indicators, geopolitical events, news releases, and investor psychology. Traditional forecasting relied heavily on statistical tools and expert-driven market intuition; however, advancements in computational modeling have transformed forecasting into a data-driven discipline.

In recent years, the integration of machine learning (ML), deep learning (DL), and hybrid computational models has become increasingly prevalent in stock market prediction systems. These techniques leverage large-scale data, extract hidden nonlinear patterns, and adapt dynamically to changing market behavior. Modern forecasting approaches are no longer limited to detecting trends but aim to perform accurate classification, regression, and automated decision-making for executing profitable trades. This shift represents a broader transformation from human-interpreted charts to sophisticated algorithmic trading systems capable of handling high-dimensional time-series data with speed and consistency.

1.2 Evolution of Technical Analysis

Technical analysis is one of the oldest methods for forecasting financial markets. Its foundation lies in the assumption that price movement reflects the collective psychology of market participants and that historical patterns tend to repeat. Tools such as candlestick formations, support-resistance levels, and indicator-based signals have been widely used for decades to evaluate potential trend reversals, continuation phases, and volatility zones. Such techniques remain popular because they do not require economic or fundamental data and instead rely solely on market-generated information.

Over time, technical analysis evolved into a structured, quantitative practice with the development of mathematical indicators such as Moving Averages, Bollinger Bands, Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), Williams, Aroon Oscillator, and Stochastic Oscillator. These indicators perform feature transformations that help traders and later machine learning models filter market noise and derive more structured predictive signals. Historical research confirms that technical indicators provide meaningful information that can aid in forecasting price direction when processed effectively.

However, while technical analysis improved the interpretability of market behavior, traditional manual usage of indicators remains subjective, prone to human bias, and limited in scalability problems that modern AI-driven systems aim to overcome.

1.3 Limitations of Classical Forecasting Models

Before machine learning methods emerged, stock forecasting relied heavily on statistical time-series models such as Moving Averages, ARIMA (Auto-Regressive Integrated Moving Average), and Holt-Winters exponential smoothing. Though effective for identifying baseline patterns, these models assume stationarity and linearity properties rarely present in real-world market behavior.

Financial time series are inherently noisy, nonlinear, chaotic, and non-stationary. Market conditions shift rapidly due to unpredictable external triggers. Classical models struggle under these conditions because they are not designed to learn evolving patterns or detect nonlinear input relationships. Research widely agrees that these approaches become insufficient as feature dimensionality increases or when multi-factor market behavior must be modeled.

Furthermore, such models often treat time series as isolated data rather than contextual sequences influenced by momentum, volatility, and behavioral patterns.

These limitations motivated the exploration of machine learning and deep learning techniques capable of capturing nonlinear and temporal dependencies in market data.

1.4 Machine Learning as a Solution

Machine learning introduced a paradigm shift in stock forecasting by enabling systems to automatically learn relationships from data rather than relying on predefined rigid formulas. ML models excel at identifying complex nonlinear interactions between features such as price history, indicators, and temporal changes making them well-suited for financial forecasting.

Traditional ML algorithms such as Support Vector Machines (SVM), Decision Trees, Random Forests, Naïve Bayes, and Logistic Regression demonstrated early success in prediction tasks. For instance, SVM models combining technical indicators such as Moving Averages, Bollinger Bands, RSI, Stochastic Oscillator, and Aroon achieved up to 77.8% prediction accuracy, demonstrating robustness especially for volatile stocks.

Similarly, research applying Decision Trees to Indian National Stock Exchange (NSE) companies reported an accuracy of 80.08%, outperforming more complex ensemble methods like Random Forests (78.8%).

However, traditional ML still required feature engineering, lacked sequential memory, and did not inherently model long-term temporal dependencies which led to the emergence of deep learning-based forecasting.

1.5 Importance of Technical Indicators

As machine learning became integrated with stock forecasting, technical indicators transitioned from trader tools to standardized computational features. Indicators such as MACD, RSI, Williams, and EMA provide derived patterns including momentum surges, trend strength, and volatility compression that offer machine learning algorithms richer, more interpretable representations of market conditions.

Multiple studies confirm that combining multiple indicators provides stronger predictive power than relying on a single signal, because each indicator captures a different aspect of price dynamics.

Indicators act as domain-optimized feature engineering, reducing noise and preventing models from learning unstable raw fluctuations. This makes them particularly effective when coupled with supervised learning algorithms and deep neural networks trained on time-series data.

1.6 Need for Hybrid Intelligent Systems

The most significant advancement in recent financial forecasting research is the emergence of hybrid models, which combine strengths of multiple architectures rather than relying on a single approach. For example, hybrid CNN-LSTM models extract short-term price patterns using CNN layers and long-term sequential behavior using LSTM, outperforming standalone models in multiple studies.

More advanced frameworks incorporate:

- Bidirectional recurrent networks
- Attention mechanisms
- Ensemble stacking
- Sentiment-enhanced features
- Reinforcement learning components

One referenced Stacked CNN-LSTM Ensemble (SCLE) integrating over 87 technical indicators and sentiment inputs demonstrated a 30% improvement in accuracy compared to a sentiment-only model.

This trend confirms that hybridization improves robustness, adaptability, and predictive accuracy especially in volatile environments.

1.7 Identified Research Gap

While existing research demonstrates significant advancements in stock prediction using ML, deep learning, and hybrid models, several gaps remain. First, a large portion of the work focuses on markets such as the NASDAQ, NYSE, and Shanghai Stock Exchange, with far less emphasis on emerging markets such as India. This creates generalization concerns, as financial behavior differs across regions due to regulation, liquidity conditions, economic stability, and trading culture.

Second, many studies prioritize prediction accuracy as the primary evaluation measure. However, high classification accuracy does not always translate to profitable or low-risk trading outcomes, especially when ignoring transaction fees, slippage, market volatility, or drawdown effects. Recent papers highlight that success metrics should incorporate profitability measures such as return on investment (ROI), maximum drawdown, and the Sharpe ratio.

Third, deep learning approaches, although powerful, are often criticized for their black-box nature, making them difficult to interpret and validate.

This lack of transparency poses challenges in regulated financial environments where decision justification is required.

Finally, while hybrid models show promise, there is limited research on systematically comparing classical ML, deep learning, and hybrid approaches specifically combined with technical analysis inputs under consistent evaluation frameworks especially using Indian market data.

These gaps collectively indicate a need for structured research that evaluates machine learning-driven technical analysis within the context of Indian market behavior while incorporating explainability and meaningful financial evaluation metrics.

1.8 Problem Statement

Based on the identified research gap, the key challenges addressed in this thesis are, traditional stock forecasting methods and standalone technical analysis approaches are limited in their ability to model nonlinear, volatile, and complex stock price behavior, particularly in emerging markets such as India. While machine learning and deep learning approaches have shown promising results, there is insufficient structured evaluation of their effectiveness when technical indicators are used as input features, especially in comparison across multiple model families and under realistic trading evaluation criteria.

This study seeks to address the problem by evaluating machine learning models for stock forecasting using technical indicators as engineered inputs, assessing their predictive capability, interpretability, and practical applicability.

1.9 Research Objectives

The main objective of this thesis is to evaluate the performance of machine learning models applied to stock technical analysis and determine their suitability for practical forecasting and decision-making.

This objective expands into the following sub-objectives:

1. To collect, preprocess, and structure historical stock data suitable for machine learning and deep learning models.
2. To compute and integrate a diverse set of technical indicators as engineered feature representations.

3. To train and evaluate multiple machine learning algorithms, including classical ML models and neural architectures.
4. To compare model performance using both statistical accuracy metrics and financially meaningful performance metrics.
5. To interpret and analyze model behavior to assess generalization capability and practical feasibility in real-world markets.

1.10 Research Questions

Aligned with the objectives, the study aims to answer the following questions:

1. Do machine learning models improve prediction performance when combined with technical analysis indicators compared to raw prices?
2. Which category of model classical machine learning, deep learning, or hybrid performs best under consistent evaluation?
3. Can models trained on historical pricing patterns generalize to unseen market data, particularly in volatile environments?
4. Do statistical accuracy metrics correlate reliably with trading performance metrics such as Sharpe ratio or return on investment (ROI)?
5. Are models based on technical analysis suitable for deployment in emerging markets such as India, or do market conditions reduce their effectiveness?

Answering these questions provides measurable insight into the viability and limitations of applying machine learning to technical stock forecasting.

1.11 Scope, Limitations, and Assumptions

The scope of this research is restricted to:

1. Historical stock price data
2. Daily time-series resolution
3. Models trained on select high-volume stocks
4. Forecast horizon limited to short-term directional or price movement prediction

Key limitations include:

1. Results may vary across different stocks, time horizons, and economic conditions.
2. The study excludes macroeconomic indicators, textual sentiment, or high-frequency tick data.
3. Interpretability challenges may persist for deeper neural architectures.
4. Model performance may degrade during extreme market volatility or structural regime shifts.
5. Assumptions include the availability of consistent market data, stable economic conditions during evaluation, and reproducibility of indicator formulas across software implementations.

1.12 Novelty and Contributions

1. The contributions of this research are summarized as follows:
2. A structured comparative study of classical ML, deep learning, and hybrid models using a uniform technical analysis-based dataset.
3. Inclusion of both accuracy metrics and financial performance metrics, aligning evaluation with real-world trading relevance.
4. Application of machine learning-driven technical analysis specifically to Indian market conditions, addressing a documented research gap.
5. Development of a replicable preprocessing and modeling pipeline for future applied research and industry use.

These contributions offer practical value for quantitative researchers, retail investors, academic researchers, and financial institutions.

Chapter 2

Literature Review

2.1 Foundations of Stock Market Forecasting and Technical Analysis

2.1.1 Introduction

Stock market forecasting has historically been grounded in rule-based trading systems derived from technical analysis and classical econometric forecasting. Before the rise of machine learning, traders relied on visually interpreted price patterns, trend signals, and moving averages to make investment decisions. Early academic research, such as that by Brock, Lakonishok, and LeBaron (1992) [1], provided one of the first systematic evaluations of technical analysis, demonstrating that rules such as Moving Average crossovers produced statistically significant returns under certain market conditions.

However, the non-linear, chaotic, and non-stationary nature of financial markets limits the consistency of purely rule-based forecasting [2]. Over time, research shifted towards formal computational modeling to improve predictive stability, reduce human subjectivity in rule interpretation, and incorporate broader feature sets including volatility, momentum, and behavioral signals. The literature shows a gradual transition from statistical forecasting models to technical indicator engineered systems, and eventually to machine learning-based predictive pipelines [3] [4].

Despite advancements, traditional forecasting approaches remain foundational and widely referenced in modern quantitative research.

2.1.2 Classical Forecasting Techniques

Early forecasting models in financial time-series analysis relied on deterministic and statistical frameworks such as Moving Averages (MA), Autoregressive Integrated Moving Average (ARIMA), and Holt Winters exponential smoothing. Moving Averages served as a baseline tool for trend smoothing and regime detection but were limited by their lag and inability to adjust to structural breaks [1].

ARIMA, formalized by Box and Jenkins (1970) [5], introduced a systematic approach to time-series modeling incorporating autoregression, differencing, and noise smoothing. Despite its widespread adoption, ARIMA requires strong assumptions of stationarity and linear structure conditions rarely satisfied in equity markets [6]. Seasonal extensions such as SARIMA and volatility-aware extensions such as GARCH further improved predictive realism but still struggled to model nonlinear market interactions [7].

Similarly, the Holt Winters method demonstrated success in modeling trend and seasonality, particularly in longer horizon forecasting, yet its performance declines during shocks, rapid trend reversals, or momentum-driven price spikes [8]. These limitations motivated the shift toward approaches capable of capturing nonlinear dependencies and adaptive real-time dynamics.

2.1.3 Technical Indicators and Traditional Trading Strategies

Technical indicators evolved as systematic computational representations of market structure, behavioral sentiment, and trend dynamics. Early research by Murphy (1999) [9] and Achelis (2001) [10] formalized indicator taxonomies, classifying them into trend-following (SMA, EMA, MACD), momentum-based (RSI, Stochastic Oscillator, Williams %R), volatility-based (ATR, Bollinger Bands), and volume-based (OBV, Chaikin Money Flow) categories.

Academic studies confirmed that these indicators filter market noise, expose turning points, and help quantify relative market strength. Brock et al. (1992) [1] demonstrated that moving average breakout and support-resistance rules could generate abnormal returns relative to random-walk baselines. Likewise, Neely et al. (2014) [11] showed that RSI and MACD exhibited predictive validity in foreign exchange markets, outperforming random walk hypothesis assumptions [3].

Later work emphasized indicator combinations rather than single-rule trading. Park and Irwin's meta-study (2007) [12] found that multi-indicator systems consistently outperform single-rule strategies, especially in volatile or trending environments. This finding aligns with modern feature engineering practice, where technical indicators

act as domain-encoded signals rather than standalone trading logic.

2.1.4 Traditional Machine Learning Models in Stock Forecasting

As financial modeling research evolved, traditional machine learning techniques emerged to bridge the gap between rule-based trading and structural statistical forecasting. Support Vector Machines (SVM), Random Forests (RF), Decision Trees (DT), and Naïve Bayes were among the earliest models applied to market prediction tasks because of their ability to capture nonlinear interactions and feature relationships.

Support Vector Machines demonstrated promising results in directional prediction when trained with technical indicator features. Studies combining Bollinger Bands, Moving Averages, Stochastic Oscillator, RSI, and Aroon Oscillator achieved accuracies up to 77.8% on volatile equity datasets [13]. Kernel-based SVM approaches remain widely used due to their ability to model nonlinear class boundaries and robustness with smaller datasets.

Decision Tree based models also gained traction for their interpretability, an important requirement in financial regulation and explainable AI contexts. A notable study applying multiple indicators to National Stock Exchange (NSE) equities reported an accuracy of 80.08%, surpassing Random Forest (78.8%) and Naïve Bayes (73.8%), showing that model simplicity can outperform ensemble complexity in structured indicator-driven environments [14].

Random Forest and Gradient Boosting extensions later improved performance stability across multiple market conditions, though interpretability trade-offs remained. These models persisted in industry applications due to their balance between predictive performance, transparency, and modest computational cost.

2.1.5 Summary

The progression from rule-based trading to statistical modeling and then to traditional machine learning reflects the increasing need to adapt forecasting methods to market noise, nonlinearity, and regime change. Classical forecasting approaches provide important theoretical foundations, while technical indicators remain integral to feature engineering pipelines. Traditional ML models represent a key transitional stage toward more advanced predictive systems and continue to offer interpretability, stability, and practical value in financial analytics.

2.2 Emerging Trends in AI-Driven Stock Forecasting

Research on stock market prediction has progressed through several distinct eras, beginning with classical statistical approaches and eventually transitioning into deep learning and, more recently, transformer-based and language-model-driven architectures. Early machine learning-based studies such as Huang et al. [13] demonstrated that supervised learning techniques could capture price dynamics more effectively than rule-based or econometric models, although they were often limited by feature dependence, poor generalization, and sensitivity to non-stationary behavior.

A major shift occurred with the introduction of recurrent neural network (RNN) variants. Hochreiter and Schmidhuber’s foundational Long Short-Term Memory model [15] established a mechanism to retain long-term dependencies in sequential data, overcoming vanishing gradient limitations common in classical RNNs. Building on this, Cho et al. [16] introduced the Gated Recurrent Unit (GRU), which simplified the gating mechanism while retaining temporal memory properties. GRUs have since become widely adopted in financial forecasting due to their computational efficiency, which is particularly beneficial when working with shorter sequences or limited historical stock data. This trend toward recurrent architectures is echoed in more recent survey-level works such as Smarte and Ho [14] and Zhu [17], who note that RNN-based models remain one of the core architectural choices in quantitative finance research.

As deep learning matured, attention shifted toward hybrid and convolutional approaches. Convolutional Neural Networks (CNNs), originally introduced by LeCun et al. [18], demonstrated strong feature extraction capabilities in structured signal domains. Their extension into financial applications, including the study by Jin [19], showed that CNN-based temporal filters could enrich predictive learning by identifying micro-patterns and localized volatility structures present within stock price movement. Hybrid CNN-LSTM systems have been especially effective at combining short-term pattern mining with longer-term sequential modeling, a trend now frequently seen in algorithmic forecasting research.

The most recent development in sequence modeling is the introduction of the Transformer architecture by Vaswani et al. [20]. Unlike recurrent models, Transformers rely solely on self-attention mechanisms to prioritize relevant time steps dynamically, enabling parallel computation and improved long-range dependency modeling. Their success in natural language processing has led to rapid adoption in finance, including applications such as sentiment-aware prediction [21], multimodal forecasting [22],

and LLM-driven trading pipelines [23]. Industry perspectives such as Cilingiroglu [24] further highlight the growing influence of context and narrative sentiment in algorithmic trading environments, reinforcing the value of attention-based systems.

Additional works such as Gupta [25], Mokhtari et al. [26], and Lu [27] emphasize that contemporary AI-driven financial systems increasingly incorporate multimodal signals, real-time market feedback, and decentralized data sources indicating a broader shift from predictive modeling toward autonomous forecasting and strategy optimization systems.

Overall, the literature demonstrates a clear methodological progression: from traditional machine learning techniques toward RNN-based deep learning, followed by hybrid convolutional-recurrent architectures, and most recently, fully attention-based and multimodal forecasting frameworks. This evolution aligns with the modeling progression explored in this project and reinforces the relevance of evaluating GRU, CNN-LSTM, and Transformer-based architectures for next-day stock price prediction under technical-analysis-driven feature frameworks.

2.3 Deep Learning Approaches

Deep learning models have become dominant due to their ability to capture temporal dependencies, non-linear relationships, and high-dimensional feature interactions. Among recurrent neural networks, LSTM architectures are the most widely adopted because they retain long-term memory and mitigate vanishing gradient issues. A study applying a four-layer LSTM on OHLC data combined with MACD, KD, RSI, and Williams %R demonstrated 83.6% prediction accuracy, with MACD performing best as an individual indicator at 76%.

CNN-based financial forecasting is also gaining traction, especially for pattern extraction and high-frequency trading. Referenced study used a one-dimensional CNN trained on 130 features for intraday classification and demonstrated competitive efficiency and predictive power.

Convolutional Neural Networks (CNNs), although originally designed for spatial pattern recognition, have been successfully adapted using 1-D filters to detect short-term sequence features. Research from Stanford applied CNNs to high-frequency intraday trading using 130 input features and demonstrated strong computational efficiency and predictive capability.

These findings highlight that deep learning outperforms traditional ML methods when sequence behavior and long-term dependencies are critical.

2.4 Hybrid Models Combining Technical Indicators and ML

A notable trend in the literature is the shift toward hybrid models that combine multiple ML or DL architectures. Hybrid CNN-LSTM models are the most prevalent approach, as they capture both short-term localized price movements (via CNN) and long-term sequential behavior (via LSTM). These hybrid models consistently outperform stand-alone LSTM or CNN architectures.

Research pushes hybrid integration further, incorporating additional layers such as bidirectional LSTM, attention mechanisms, and ensemble stacking. One cited Stacked CNN-LSTM Ensemble (SCLE) integrated 87 technical indicators with sentiment-based input and achieved 30% improved prediction accuracy over sentiment-only systems, showing strong evidence that multimodal approaches yield superior results.

Reinforcement learning adds another dimension by focusing not on prediction accuracy but on portfolio profitability and Sharpe ratio optimization.

2.5 Reinforcement Learning Approaches

Reinforcement Learning (RL) presents a fundamentally different approach by learning optimal trading strategies (buy/sell/hold) instead of predicting future prices. RL models are explicitly rewarded for profit-maximization under volatility and transaction constraints.

A referenced study using Advantage Actor-Critic (A2C) demonstrated improved risk-adjusted performance with a Sharpe ratio of 0.13, outperforming the DJI index baseline with a negative Sharpe ratio.

Such results suggest that RL may resolve the disconnect between high statistical forecasting accuracy and actual trading profitability one of the most critical challenges in ML-based stock prediction.

2.6 Comparison Between different Model and research

Table 2.1: Summary of Existing Machine Learning, Deep Learning, and Hybrid Approaches in Stock Market Forecasting

Study / Reference	Technical Indicators Used	Methodology and Main Findings
Huang et al., 2022 [13]	Standard price-based indicators and derived historical features	Compared classical machine learning models for global stock markets. Found that ML models outperform traditional statistical forecasting but struggle during high volatility.
Jin, 2024 [19]	CNN-derived feature maps + classical stock indicators	Proposed a hybrid CNN-based deep learning model to extract local patterns from technical indicators. Demonstrated improved performance over shallow networks.
Smarte & Ho, 2021 [14]	Broad survey of technical indicators including RSI, MACD, EMA, Bollinger Bands	Concluded that deep learning models outperform SVM/Random Forest when time dependency and financial noise are considered.
Khanpuri et al., 2024 [21]	Indicators integrated with textual sentiment signals	Combined sentiment from fundamental data with machine learning classifiers. Demonstrated that hybrid sentiment + indicator models outperform indicator-only systems.
Huang et al., 2025 [22]	Mixed quantitative indicators + deep learning feature extractions	Surveyed the evolution of modern quantitative investment from LSTM to LLM-based pipelines. Highlighted multimodal architectures as emerging state-of-the-art.
Cao et al., 2024 [23]	No traditional indicators; used contextual embeddings from market text	Applied LLMs for market forecasting. Demonstrated that language-driven forecasting outperforms numerical-only models under news-driven volatility.
Balasubramanian et al., 2024 [28]	Experimental biometric signals alongside price indicators	Showed that non-price features such as emotional states may correlate with market responses, suggesting behavioral-finance-based augmentation.
Mokhtari et al., 2021 [26]	Price-based indicators with reinforcement learning signals	Discussed the role of AI in automated trading. Concluded that reinforcement learning enhances long-term portfolio performance over static ML predictors.

Study / Reference	Technical Indicators Used	Methodology and Main Findings
Zhu, 2025 [17]	Survey of indicator-based forecasting strategies	Highlighted that technical-analysis-driven ML systems remain effective when paired with deep architectures such as LSTM or transformers.

2.7 Key Findings, Strengths, and Limitations

Across surveyed research, key themes emerge:

Dimension Findings - Strengths ML/DL models outperform statistical models, hybrid approaches produce superior results

Insights Technical indicators significantly enhance ML performance, multimodal data boosts accuracy

Limitations - Models often lack interpretability; accuracy does not always equate to profitability

Additionally, many studies report performance solely through predictive accuracy, which does not account for slippage, transaction costs, or real-market risk exposure. Researchers now argue for financial evaluation metrics such as Sharpe ratio, max drawdown, and ROI.

Table 2.2: Comparison of machine learning, hybrid deep learning, and emerging AI approaches for stock market prediction.

Study / Approach	Technical Indicators Used	Algorithm / Model	Key Results / Findings
Huang et al., 2022 [13]	Historical price-derived indicators (general TA-based dataset)	Classical ML models (SVM, RF, Logistic Regression)	Machine learning models outperform statistical baselines but suffer from instability under volatile market regimes.
Jin, 2024 [19]	CNN-generated feature maps + TA-based indicator inputs	Hybrid CNN-based deep learning system	CNN-based feature extraction improves modeling of local temporal patterns and yields better accuracy than traditional shallow NN architectures.
Smarte & Ho, 2021 [14]	RSI, MACD, EMA, Bollinger Bands, ADX (survey dataset)	Comparative ML/DL review	Concluded that deep learning (LSTM/GRU) consistently outperforms SVM/Decision Trees when sequential dependencies and market noise are considered.

Study / Approach	Technical Indicators Used	Algorithm / Model	Key Results / Findings
Khanpuri et al., 2024 [21]	Technical indicators + fundamental textual sentiment	Sentiment-based ML pipeline	Hybrid indicator + sentiment models outperform indicator-only models when predicting post-news volatility and direction.
Mokhtari et al., 2021 [26]	Historical price and technical-analysis signals	Reinforcement Learning (A2C, DDPG)	RL-based systems outperform static ML models in long-term trading environments; demonstrated positive risk-adjusted returns relative to baseline benchmarks.
Cao et al., 2024 [23]	No indicators; text-based embeddings extracted from news and reports	LLM-based forecasting	LLM models respond well to news-driven volatility and show stronger adaptability compared to purely numerical models.
Huang et al., 2025 [22]	Mixed quantitative TA indicators + deep embeddings	Survey of DL → LLM evolution	Identifies transformer and LLM-based frameworks as the next major stage in quantitative finance research.
Zhu, 2025 [17]	Mixed indicator-based forecast scenarios	Review of AI models for forecasting	Reinforces the need for hybrid pipelines; concludes combined sentiment + price features outperform single-signal systems.
Gupta, 2024 [25]	Price-driven TA signals + strategy metadata	AI-assisted quantitative trading	Demonstrated improved algorithmic trading performance when AI was integrated into execution and portfolio optimization strategies.

2.8 Evaluation Challenges, Research Gaps and Inconsistencies

Across reviewed studies, accuracy is the most reported metric; however, accuracy alone does not reliably reflect trading profitability. Multiple studies highlight the need for more holistic metrics such as annualized return, max drawdown, and Sharpe ratio to reflect real-world trading quality.

Interpretability remains another challenge, especially in regulated sectors where explainable decision-making is critical. The literature emphasizes that opaque deep learning models face adoption barriers when their behavior cannot be justified.

Finally, overfitting remains persistent due to the chaotic and non-stationary properties of financial data, requiring adaptive retraining, rolling validation, and multi-market testing strategies.

Despite progress, several gaps remain:

- Overfitting persists due to non-stationary data.
- Limited generalizability across markets, time periods, and asset classes.
- Few models evaluate real trading performance rather than classification accuracy.
- Indian markets remain underexplored relative to U.S. and Chinese datasets.
- RL-based approaches are promising but still sparse and experimental.

2.9 Alignment with the Present Research

The literature demonstrates a clear progression from classical technical analysis toward machine learning-driven forecasting systems. Traditional ML models offer interpretability, while deep learning models especially hybrid CNN-LSTM architectures provide superior predictive capability by capturing multi-dimensional patterns in historical price signals. Reinforcement learning represents the next frontier, enabling models not just to predict markets but to act intelligently within them.

The works reviewed confirm that machine learning methods especially hybrid and deep learning-based approaches yield superior results when combined with carefully engineered technical indicators. However, the gap in generalizable, explainable, and India-focused ML-based forecasting highlights the need for a model specifically designed to work with Indian market behavior.

This thesis positions itself at this intersection: applying modern ML techniques integrated with technical indicators, evaluating not only accuracy but also potential trading practicality on Indian market securities.

Chapter 3

Methodology

3.1 Overview

This chapter describes the methodological framework used to build, train, and evaluate machine learning models for predicting next-day closing prices of Indian stocks. The approach includes data acquisition, preprocessing, feature engineering using technical indicators, model architecture design, training procedures, and the final prediction and deployment pipeline. The modelling strategy includes both single-stock models and generalized multi-stock architectures to evaluate how model scope influences predictive performance.

3.2 System Workflow

The overall methodology follows the structured workflow below:

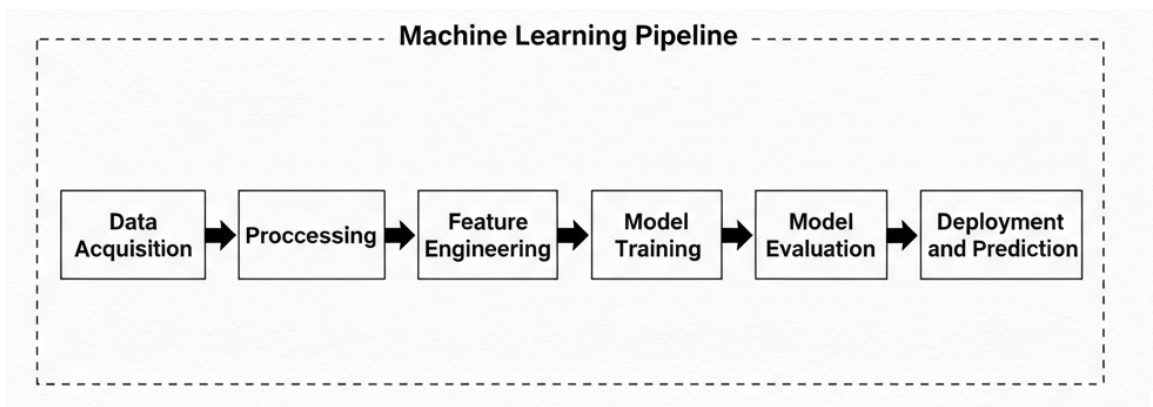


Figure 3.1: System Flow

3.3 Data Acquisition

Historical stock data is collected using the Kite Connect API, which provides daily OHLCV (Open, High, Low, Close, Volume) records for Indian equity markets. Three data scopes are utilized:

Table 3.1: Dataset Categorization Based on Usage and Scope

Dataset Type	Usage	Scope
Company-specific dataset	LSTM & GRU individual models	Single company, 2-year window
NIFTY-based dataset	CNN-LSTM and baseline analysis	NIFTY 50 and NIFTY 200 constituents
Full-market dataset	Transformer-based architecture and cross-learning	All available NSE-listed companies

All datasets use two years of historical records, sampled at one trading day frequency. Missing trading days (holidays, suspensions) are handled by forward-fill as part of preprocessing.

3.4 Data Preprocessing

Preprocessing ensures consistency, numeric uniformity, and suitability for machine learning. Key steps include:

3.4.1 Handling Missing Values

Financial time-series datasets often contain missing or irregular values due to weekends, market holidays, low liquidity periods, or interruptions in data collection. Since machine learning models require complete and continuous input sequences to learn temporal dependencies, the preprocessing stage included systematic handling of missing and inconsistent data. The goal of this step was to preserve the chronological structure of the dataset while minimizing artificial distortion of historical behavior.

Missing values encountered in this project were primarily associated with two scenarios: (1) non-trading days where the market was closed and no new data entries existed, and (2) occasional missing volume entries resulting from low-activity sessions or incomplete API retrieval.

Handling Non-Trading Days

Since stock markets do not operate during weekends and public holidays, gaps naturally appear in raw historical price data. Instead of removing these entries or artificially inserting synthetic timestamps, a forward-fill strategy was applied to maintain consistency within the time index. Forward filling ensures that the most recent available market value is propagated to the next missing observation, formalized as:

$$x_t = \begin{cases} x_t, & \text{if value exists} \\ x_{t-1}, & \text{otherwise} \end{cases} \quad (3.1)$$

This approach preserves trend continuity without introducing computational artifacts. Since the forecasting window is daily and does not rely on intraday variation, forward filling provided a stable and contextually accurate method for gap handling.

Missing Volume Replacement

In rare instances, isolated gaps in the `Volume` field were detected. Volume values play an important role in financial modeling, as they reflect market participation and liquidity. Removing or forward filling volume may obscure meaningful behavioral dynamics, particularly during sudden price swings driven by abnormal trading pressure. To address this, missing volume values were replaced using a 7-day rolling mean:

$$Volume_t = \frac{1}{7} \sum_{i=1}^7 Volume_{t-i} \quad (3.2)$$

This method ensured a smooth and context-aware replacement rather than assigning an arbitrary constant or repeating prior values. The rolling mean preserves short-term variability while avoiding sudden artificial spikes.

Indicator-Related Missing Rows

Some derived feature calculations, such as moving averages or volatility estimators, require a minimum historical window before a valid output can be produced. This results in initial rows of the dataset lacking values for engineered features. Instead of performing interpolation or backfilling—which could introduce bias—these incomplete rows were removed entirely:

$$D_{clean} = \{x \in D \mid \text{all features at } t \text{ are non-null}\} \quad (3.3)$$

Since dataset length remained sufficient after removal and no temporal leakage was introduced, this approach ensured data consistency during feature modeling and training.

Rationale for the Missing Data Strategy

The combined approach of forward filling, rolling mean substitution, and removal of incomplete engineered rows was selected to:

- preserve authentic historical trends,
- prevent artificial volatility distortions,
- avoid information leakage,
- maintain training stability,
- ensure that the sequential nature of the dataset remained intact.

In time-series forecasting, the quality of preprocessing directly affects model convergence and accuracy. The strategies implemented in this project ensured continuity of data without introducing artificial dynamics or modifying underlying market behavior patterns, thereby supporting more reliable model learning during subsequent stages.

3.4.2 Normalization

Since machine learning models operate more effectively on normalized sequences, the following scaling methods are used:

Min-Max scaling for time-dependent features:

$$x_{\text{scaled}} = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (3.4)$$

Target variable (next closing price) scaled using the same transformer to maintain data consistency.

Each company's scaling parameters are stored individually to allow inverse transformation during inference.

3.5 Feature Engineering

Feature engineering plays a crucial role in preparing financial time-series data for machine learning models. Raw stock market data contains valuable information;

however, it is not always presented in a form suitable for model learning. The purpose of feature engineering is to transform raw values into structured inputs that enhance pattern recognition and improve predictive performance. In this project, feature engineering focused on restructuring historical price and volume data into supervised learning format, generating lagged dependencies, scaling features for training stability, and preparing the data for sequence-based neural network architectures.

3.5.1 Raw Feature Selection

The raw data retrieved consisted of daily Open, High, Low, Close, and Volume (OHLCV) values for each stock. These values represent the fundamental inputs required to characterize market movement. In this project, these five variables were selected as the baseline feature set because they directly correspond to observable trading activity and remain consistent across companies and time periods.

The feature set is defined as:

$$X_t = \{Open_t, High_t, Low_t, Close_t, Volume_t\} \quad (3.5)$$

This ensured a uniform representation of stock behavior and provided the foundational structure upon which additional transformations were applied.

3.5.2 Temporal Windowing

Since stock market data is sequential, the learning task was reframed from a single-point prediction to a supervised time-series problem. A sliding window mechanism was introduced to create input-output pairs, where multiple past observations are used to forecast the next value:

$$X = [X_{t-n}, X_{t-n+1}, \dots, X_{t-1}] \rightarrow y = Close_t \quad (3.6)$$

In this project, a window size of 20 trading days was selected based on empirical testing and industry practice. This window length allowed the model to observe short-term trend formation and volatility behavior without inducing excessive computational load or overfitting risk.

3.5.3 Lagged Variables

Lag-based values were implicitly generated through the windowing process. By exposing the model to historical values as ordered sequences, the neural architectures

were able to infer temporal relationships such as:

- Price acceleration or deceleration
- Recent volatility conditions
- Volume shifts related to market sentiment

This sequence representation helped the models learn time-dependent structure in the data more effectively than using only the most recent price.

3.5.4 Scaling and Normalization

Financial data often varies significantly in magnitude across stocks due to differences in sector, price range, liquidity, and volatility regimes. To ensure numerical stability and improve gradient behavior during training, Min-Max scaling was applied:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (3.7)$$

Scaling was performed independently for each stock to preserve relative relationships within a sequence. The inverse transformation was retained to enable final predictions to be mapped back to real currency values during model evaluation and deployment.

3.5.5 Train-Test Structuring

The final step in feature engineering involved restructuring the processed dataset into training and testing segments while maintaining temporal order. Unlike shuffled machine learning tasks, stock prediction requires chronological integrity to prevent target leakage. Therefore, the data was divided as:

$$D = D_{\text{train}} \cup D_{\text{test}}, \quad \text{with } t_{\text{train}} < t_{\text{test}} \quad (3.8)$$

This ensured that only historical values contributed to model training, aligning the process with realistic forecasting constraints.

3.5.6 Summary of Feature Engineering Process

Through the combination of raw feature selection, temporal restructuring, scaling, and preparation into supervised sequential format, the dataset was transformed into a suitable structure for neural network modeling. These engineered features enabled the

models to better recognize temporal dependencies, smoothing patterns, and sequential price dynamics essential for short-term stock prediction.

3.6 Technical Indicators

Technical indicators are mathematical transformations of historical price and volume data used to extract meaningful market patterns and assist in short-term forecasting. In financial modeling, indicators may help reveal momentum, volatility, or market sentiment, which is often not directly observable from raw price values. In this project, a select group of widely used indicators was implemented and evaluated to determine whether engineered financial signals improve model learning and predictive stability.

The indicators used fall into four major categories: trend-based, momentum-based, volatility-based, and volume-based indicators. The following subsections describe each indicator mathematically and conceptually, along with its motivations, advantages, limitations, and observed relevance within this project's modeling framework.

3.6.1 Exponential Moving Average (EMA)

The Exponential Moving Average (EMA) is a trend-following indicator that assigns more weight to recent price values, making it more responsive to recent movements compared to a simple moving average. In this project, two commonly used EMAs, with window lengths 12 and 26, were included.

The EMA is defined as:

$$EMA_t = \alpha \cdot P_t + (1 - \alpha) \cdot EMA_{t-1} \quad (3.9)$$

where:

- P_t is the closing price at time t
- $\alpha = \frac{2}{n+1}$ is the smoothing factor
- n is the lookback period (12 or 26 in this study)

Advantages:

- Reacts faster to trend changes than simple moving averages
- Helps highlight market direction and momentum
- Can provide meaningful structure when used with machine learning features

Limitations:

- May introduce noise during volatile market periods
- Requires calibrated window lengths depending on market characteristics

Role in This Project: The EMA was intended to help the models detect smoother directional patterns compared to raw price sequences. In practice, the EMA contributed most effectively in larger datasets, where its smoothing characteristics complemented the learning process of more complex models like LSTM and CNN-LSTM.

3.6.2 Relative Strength Index (RSI)

The Relative Strength Index (RSI) is a momentum oscillator used to measure the speed and magnitude of recent price changes and identify potential overbought or oversold conditions.

The RSI is computed as:

$$RS = \frac{\text{Average Gain}}{\text{Average Loss}} \quad (3.10)$$

$$RSI = 100 - \left(\frac{100}{1 + RS} \right) \quad (3.11)$$

where gains and losses are smoothed using an exponential approach over a default 14-day period.

Advantages:

- Captures short-term momentum and reversal behavior
- Useful for detecting buying or selling pressure

Limitations:

- Can produce false signals during trend-driven markets
- Highly sensitive to short-term volatility

Role in This Project: RSI helped introduce oscillatory behavior into the feature space, offering counterbalance to trend-heavy predictors. While effective for larger and diverse datasets, its influence was limited in single-stock training where raw OHLC patterns already contained equivalent momentum signals.

3.6.3 Average True Range (ATR)

The Average True Range (ATR) is a volatility indicator designed to measure the degree of price range expansion or contraction over time. It is computed using the True Range (TR), defined as:

$$TR_t = \max \left(\begin{array}{l} H_t - L_t, \\ |H_t - C_{t-1}|, \\ |L_t - C_{t-1}| \end{array} \right) \quad (3.12)$$

The ATR is calculated using:

$$ATR_t = EMA(TR_t, 14) \quad (3.13)$$

Advantages:

- Helps quantify market volatility
- Adds information not present in raw closing values

Limitations:

- Does not indicate price direction
- Can amplify noise when used in small datasets

Role in This Project: ATR was included to expose volatility dynamics to the prediction models. The indicator was most beneficial when used in larger datasets, although its contribution was minimal in single-stock regression tasks.

3.6.4 On-Balance Volume (OBV)

On-Balance Volume (OBV) is a cumulative volume-based indicator designed to track buying and selling pressure.

$$OBV_t = \begin{cases} OBV_{t-1} + V_t, & \text{if } C_t > C_{t-1} \\ OBV_{t-1} - V_t, & \text{if } C_t < C_{t-1} \\ OBV_{t-1}, & \text{otherwise} \end{cases} \quad (3.14)$$

where V_t represents the traded volume.

Advantages:

- Captures volume shifts preceding price movement

- Can reveal accumulation or distribution patterns

Limitations:

- Highly sensitive to volume spikes
- May not generalize well across low-liquidity stocks

Role in This Project: OBV served as a complementary feature for models trained across multiple companies, providing volume dynamics that pure price-based features could not convey.

3.6.5 Summary of Indicator Contribution

In this project, indicator-based feature sets provided mixed results. While they improved model generalization in larger datasets, simpler raw OHLCV features performed better for single-stock forecasting tasks. This outcome suggests that engineered indicators have higher value when the training scope involves diverse or multi-instrument learning rather than narrow market contexts.

3.7 Windowing Strategy

Since predictions are based on sequential input, a sliding window approach is applied:

Input window: 20 most recent trading days

Output horizon: Next-day closing price (single-step forecasting)

This creates training pairs of the form:

$$X = [\text{features for days } t - 19 \text{ to } t]$$

$$y = \text{closing price at day } t + 1$$

This ensures the model learns short-term memory patterns relevant to near-term forecasting.

3.8 Model Training and Validation

Training is carried out with:

Train/Test split: 80% / 20%

Validation method: time-series split (no shuffling)

Callbacks: early stopping and learning rate reduction

Loss curves and prediction visualizations are generated for model comparison.

3.9 Prediction and Deployment Pipeline

Once trained, models are exported and integrated into a lightweight prediction service.

Deployment flow:

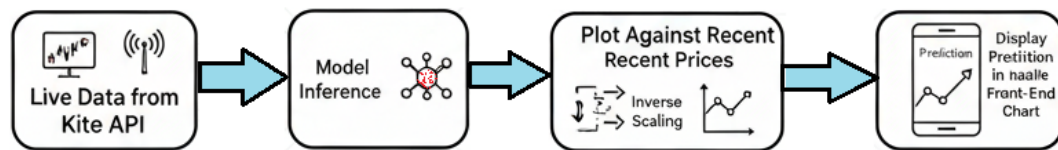


Figure 3.2: Architecture of Data Flow.

Predictions are plotted alongside the last 3 months of market data for visual validation.

Chapter 4

Experimental Setup

4.1 Overview

This chapter presents the experimental setup used to train, evaluate, and compare the machine learning models designed for next-day stock closing price prediction. It includes details about the computing environment, software dependencies, dataset configuration, training parameters, performance metrics, and model validation strategy. The goal of the experimental setup is to ensure consistency across all experiments and provide a reproducible framework for future extensions.

4.2 Hardware Configuration

All experiments were conducted on a system equipped with the following hardware:

Table 4.1: Recommended Hardware Specifications

Component	Specification
Processor	AMD Ryzen 5600G or equivalent multi-core CPU
RAM	16 GB DDR4
Storage	SSD (for faster dataset access and training runs)
GPU (optional)	AMD RX580 (Limited training Support)

While basic models such as LSTM and GRU can run efficiently on CPU, GPU acceleration significantly reduced training time for larger models such as the Transformer and CNN-LSTM hybrid.

4.3 Software Stack

The implementation was performed using the following software tools:

Table 4.2: Project Technology Stack

Category	Libraries / Tools
Programming Language	Python
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
Machine Learning	scikit-learn
Deep Learning	TensorFlow / Keras
API Data Access	Kite Connect API
Deployment	Custom front-end interface for visualization

4.4 Dataset Preparation

The dataset consists of two years of daily historical stock data for companies included in the NIFTY50, NIFTY200, and broader NSE universe. Data preprocessing steps included:

- Forward-filling missing values for non-trading days
- Normalizing features using Min-Max scaling
- Computing technical indicators
- Structuring the final dataset into supervised learning format

A sliding window of the previous 20 trading days was used as input to predict the next day closing value.

4.5 Train-Test Split Strategy

To preserve chronological order and prevent data leakage, random shuffling was avoided.

Validation was managed via a time-series validation approach rather than k-fold cross validation.

Table 4.3: Dataset Splitting Strategy

Dataset Type	Split
Training Set	80% of historical sequence
Testing Set	Remaining 20%

4.6 Hyperparameter Configuration

Multiple models were trained using consistent baseline hyperparameters with fine-tuning performed through iterative experimentation. The following table summarizes the core parameters:

Table 4.4: Model Hyperparameters

Parameter	Value / Range
Epochs	50–200 (early stopping applied)
Batch Size	32–64
Optimizer	Adam
Learning Rate	0.001 (adaptive reduction on plateau)
Loss Function	Mean Squared Error (MSE)

Activation functions consisted of ReLU for hidden dense layers and linear activation for the output layer.

4.7 Model-Specific Training Setup

Each model required additional configuration:

4.7.1 LSTM And GRU Models

- Sequential input with shape (20 timesteps * features)
- Single hidden recurrent layer with dropout to reduce overfitting

4.7.2 CNN-LSTM Model

- Initial convolution layer for local pattern extraction
- LSTM layer for temporal sequence learning

4.7.3 Transformer Model

- Multi-head self-attention layers
- Positional encoding to preserve temporal structure

Transformer models required the longest training time due to complexity.

4.8 Evaluation Metrics

The following regression metrics were used to evaluate model performance:

Table 4.5: Model Evaluation Metrics

Metric	Description
MAE	Measures average magnitude of error (Mean Absolute Error)
RMSE	Penalizes larger errors more heavily (Root Mean Squared Error)
MAPE	Expresses prediction error as a percentage (Mean Absolute Percentage Error)
R ² Score	Measures goodness of fit (Coefficient of Determination)

Additionally, prediction plots were generated to visually compare predicted vs. actual closing prices.

4.9 Deployment Testing

After training, the best-performing models were integrated into a lightweight deployment pipeline. Real-time prediction testing involved:

- Fetching latest daily closing values through Kite API
- Preprocessing the data using stored scaling and feature generation rules
- Running inference using trained models
- Overlaying predictions on front-end interactive charts

A rolling evaluation over the last three months of data was also performed to simulate real-world model behavior.

Chapter 5

Model Details

5.1 Gated Recurrent Unit (GRU) Models

5.1.1 Introduction to GRU Architecture

The Gated Recurrent Unit (GRU) is a type of recurrent neural network (RNN) designed to process sequential or time-dependent data by retaining useful temporal patterns during training. Unlike feed-forward neural networks, GRUs maintain memory across input steps, making them suitable for tasks where the past influences the future—such as financial time-series forecasting. GRUs achieve this by using gating mechanisms that determine how much historical information should be retained or forgotten during sequence processing.

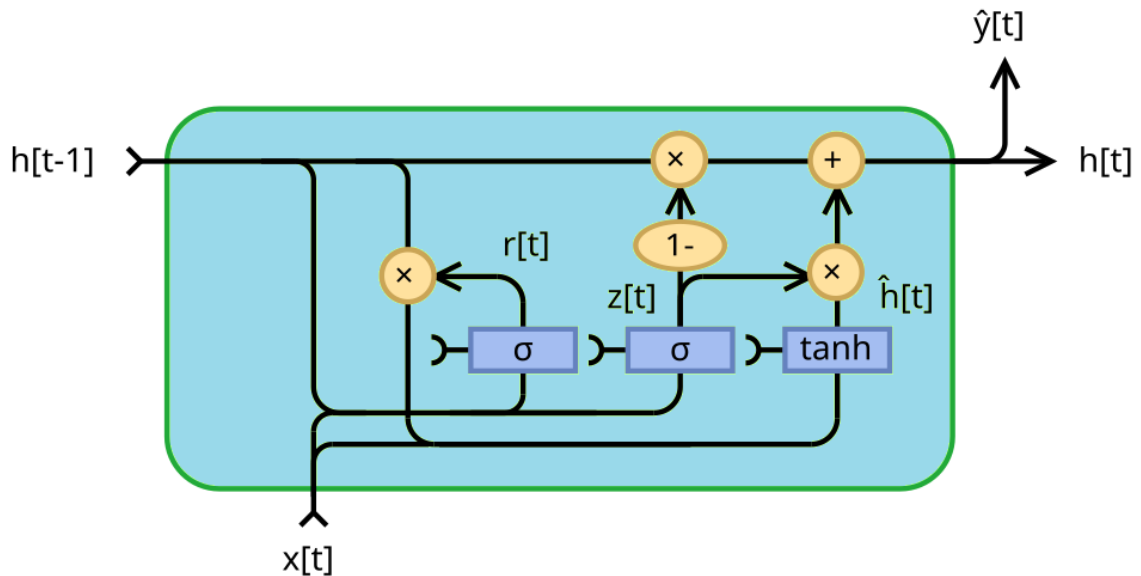


Figure 5.1: Architecture of the GRU block.

GRUs contain two primary gates:

- Update Gate: Controls how much of the past information should remain relevant for the next output.
- Reset Gate: Decides how much of the previous memory should be ignored when integrating new input signals.

This gated mechanism allows GRUs to learn long-term dependencies efficiently while avoiding vanishing or exploding gradients commonly seen in standard RNNs.

5.1.2 Why GRUs Are Used in Time-Series Forecasting

GRUs are widely used for sequential modeling due to their ability to:

- Generalize well with shorter sequences and limited training samples
- Capture temporal relationships in noisy or irregular data
- Model nonlinear patterns without manually engineered lag features

In stock price forecasting, this is particularly useful because market patterns evolve gradually, yet abrupt fluctuations occur. GRUs strike a balance between memory and responsiveness, making them suitable for short-term prediction tasks such as next-day closing price forecasting.

5.1.3 Mathematical Model for GRU

$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$	Reset Gate
$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$	Update Gate
$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$	Candidate Hidden State
$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	Final Hidden State

Symbol	Description
x_t	Input vector at time t .
h_{t-1}	Hidden state from the previous time step $t - 1$.
h_t	Final hidden state (output) at time t .
Continued on next page	

Symbol	Description
W_i, U_i, b_i	Weight matrices (W for input, U for recurrent state) and bias vectors to be learned.
$\sigma(\cdot)$	The sigmoid activation function ($\frac{1}{1+e^{-x}}$), which outputs values between 0 and 1.
$\tanh(\cdot)$	The hyperbolic tangent activation function , which outputs values between -1 and 1.
$\odot$	The Hadamard product (element-wise multiplication).

Table 5.1: Key Notations for Gated Recurrent Unit (GRU) Equations

5.1.4 Advantages and Disadvantages of GRUs Category Details

5.1.5 Role of GRU in This Project

GRU models were selected as a baseline and performance benchmark for stock prediction in this project. Since the dataset per individual company consists of approximately two years of daily values, the model size and structure had to balance complexity with generalization ability.

GRUs performed particularly well when trained using only raw OHLCV data. In this case, the model adapted efficiently to the stock's behavioral characteristics and produced stable and accurate predictions. However, this also made the model highly stock-specific, meaning generalization to unseen companies was limited.

When trained on broader datasets such as NIFTY50 or NIFTY200, the GRU model benefited from the additional variability but showed slight performance degradation unless tuned with regularization methods such as dropout.

5.1.6 Trained GRU Model Configurations

Model 1 — GRU with Only Stock Data (No Indicators)

This version used only the raw OHLCV features as input and trained for 60 epochs. It consisted of a single GRU layer followed by dropout and dense layers to generate the output.

This lightweight structure allowed fast convergence and produced strong performance for individual stock prediction, especially when dataset size was limited.

Category	Details
Advantages	• Faster training compared to LSTM due to fewer parameters • Performs well with limited data • Avoids vanishing gradient problem better than simple RNNs • Works effectively without feature engineering in many cases
Disadvantages	• Slightly less expressive than LSTMs for complex long-range patterns • May overfit when trained only on one stock without regularization • Interpretability remains limited compared to simpler models
Shortcomings in Context	• Requires careful hyperparameter tuning to avoid memorizing past trends • Sensitivity to scaling and outlier behavior in noisy financial datasets

Table 5.2: Assessment of Model Characteristics (likely GRU/Simple RNN variant)

Model 2 — GRU with Technical Indicators

The second configuration incorporated engineered features, including trend, momentum, volatility, and volume-based indicators:

This version used a deeper architecture with two stacked GRU layers. It was also trained for 60 epochs.

Although the model improved generalization in multi-stock settings, it occasionally underperformed the non-indicator version for single-stock forecasting. This suggests that engineered indicators can introduce noise or redundancy when training data is limited.

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 64)	13,632
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Table 5.3: Model Architecture Summary

Type	Indicator	Description
Trend	EMA (12, 26)	Exponential moving averages
Momentum	RSI (14)	Relative Strength Index
Volatility	ATR (14)	Average True Range
Volume-based	OBV	On-Balance Volume

Table 5.4: Summary of Technical Indicators

Layer (type)	Output Shape	Param #
gru_1 (GRU)	(None, 20, 128)	53,760
dropout_1 (Dropout)	(None, 20, 128)	0
gru_2 (GRU)	(None, 64)	37,248
dropout_2 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

Table 5.5: Stacked GRU Model Architecture Summary

5.1.7 Summary of GRU Model Performance

Key takeaways from the GRU experimentation include:

- GRU models offer the best speed-to-accuracy balance among tested architectures.
- Using only raw OHLCV values yielded surprisingly strong results, especially for stock-specific forecasting.
- With technical indicators included, GRUs generalized better across multiple stocks but required careful tuning.
- GRUs represent a stable baseline model and provide a practical starting point for stock market forecasting.

5.2 Long Short-Term Memory (LSTM) Model

5.2.1 Introduction to LSTM Architecture

The Long Short-Term Memory (LSTM) network is an extension of recurrent neural networks designed to effectively learn patterns in sequential data while avoiding the vanishing and exploding gradient limitations present in standard RNNs. LSTMs achieve this through a gated structure consisting of input, forget, and output gates, which regulate how information flows forward through time and how much past information is retained.

The LSTM cell introduces an internal memory state (cell state) that stores useful information across longer time spans than traditional RNNs or even GRUs in some cases. This makes LSTM particularly suitable for financial time-series forecasting, where historical patterns and gradual momentum shifts can influence short-term price outcomes.

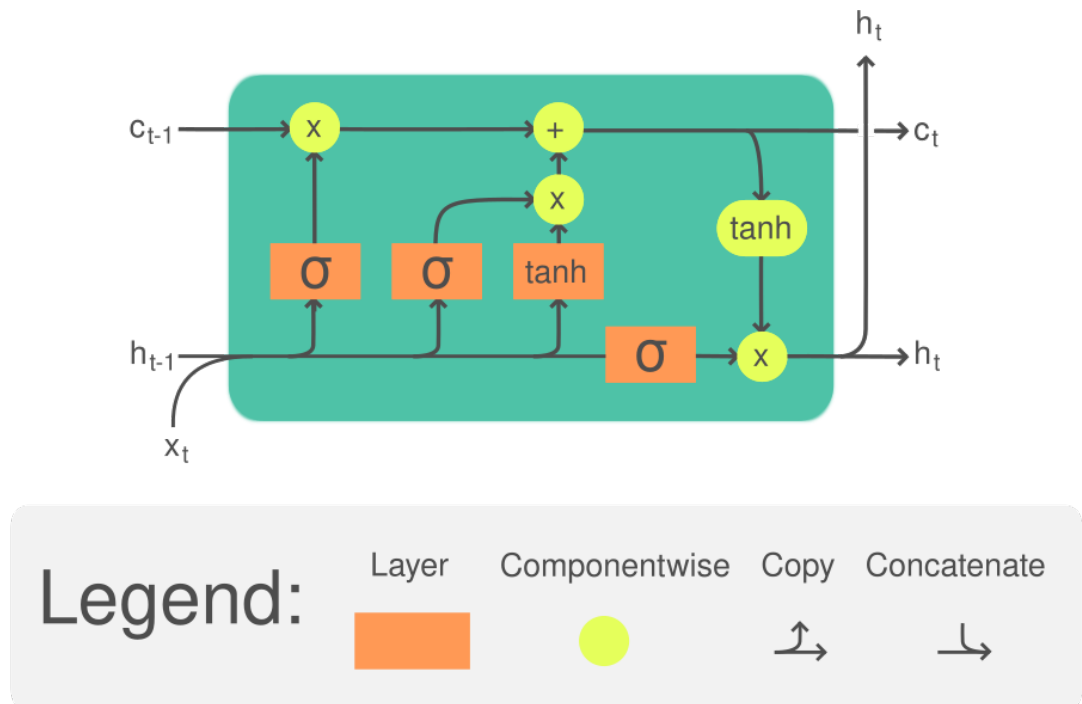


Figure 5.2: Architecture of the LSTM block.

5.2.2 Why LSTM Is Used for Time-Series Forecasting

LSTM networks are widely adopted in forecasting applications because they are capable of learning temporal dependencies, handling non-linear input behavior, and capturing longer patterns in data such as trend formation and reversal sequences. In the context of stock prediction, price movement is rarely independent from its history; trends, oscillations, and volatility cycles evolve over time. LSTM models are designed to retain this form of information and are therefore an appropriate choice for next-day price forecasting.

5.2.3 Mathematical Model for LSTM

$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$	Forget Gate
$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$	Input Gate
$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$	Output Gate
$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$	Candidate Cell State
$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	Updated Cell State
$h_t = o_t \odot \tanh(c_t)$	Final Hidden State (Output)

Symbol	Description
x_t	Input vector at time step t . Typically contains the feature values (e.g., OHLCV and/or indicators) for the current day.
h_{t-1}	Hidden state from the previous time step $t - 1$. Encodes past information propagated through the sequence.
h_t	Final hidden state (output) at time step t , used either directly for prediction or passed to subsequent layers.
c_{t-1}	Previous cell state at time step $t - 1$, representing long-term memory.
c_t	Updated cell state at time step t , storing both retained and newly added information.
f_t	Forget gate vector at time t , controlling which information from c_{t-1} is retained ($f_t \approx 1$) or discarded ($f_t \approx 0$).
i_t	Input gate vector at time t , controlling how much of the candidate state $\tilde{c}_t$ is written into the cell state.
Continued on next page	

Symbol	Description
o_t	Output gate vector at time t , controlling how much of the internal cell state c_t is exposed to the hidden state h_t .
$\tilde{c}_t$	Candidate cell state at time t computed from current input and previous hidden state, before being filtered by the input gate.
W_f, W_i, W_o, W_c	Weight matrices applied to the input x_t for the forget, input, output, and candidate cell computations respectively.
U_f, U_i, U_o, U_c	Recurrent weight matrices applied to the previous hidden state h_{t-1} for the forget, input, output, and candidate cell computations.
b_f, b_i, b_o, b_c	Bias vectors associated with the forget, input, output, and candidate cell transformations.
$\sigma(\cdot)$	Sigmoid activation function, $\sigma(x) = \frac{1}{1+e^{-x}}$, squashing values into the range $(0, 1)$ and used for gating.
$\tanh(\cdot)$	Hyperbolic tangent activation function, mapping values to $(-1, 1)$ and used to create bounded candidate and exposed states.
$\odot$	Hadamard (element-wise) product between two vectors or matrices.

Table 5.6: Key notations and equations for the Long Short-Term Memory (LSTM) network

5.2.4 Advantages and Disadvantages of LSTM

5.2.5 Role of LSTM in This Project

LSTM models were included to evaluate how memory-retaining architectures behave across different dataset scales and feature complexity. The LSTM models trained in this project showed strong results both in single-company and multi-company scenarios.

When trained on only OHLCV data, the LSTM model demonstrated smooth prediction behavior and closely followed price movements. Similar to the GRU model, this setup performed exceptionally well for stock-specific forecasting, although this accuracy partially stems from the model's capacity to memorize the stock's internal movement pattern.

When trained using additional technical indicators and broader datasets such

Category	Details
Advantages	• Able to learn long-term dependencies more effectively than standard RNNs or GRUs • Performs well on noisy sequences where patterns unfold gradually • Stable training dynamics for multi-stock datasets • Less prone to catastrophic forgetting
Disadvantages	• Higher computational cost due to larger number of parameters • Longer training time compared to GRU • May overfit when data availability is limited unless regularization is applied
Project Context Limitations	• When trained on a single stock, LSTM models may learn stock-specific behavior too well, reducing cross-stock generalization • Performance benefit compared to GRU is noticeable mainly when the dataset size increases

Table 5.7: Assessment of LSTM Model Characteristics

as NIFTY50 and NIFTY200, the LSTM architecture generalized better than GRU, particularly in diverse market conditions. This suggests that the LSTM structure is better suited for representing multi-phase temporal patterns that occur across companies and industries.

5.2.6 Trained LSTM Model Configurations

LSTM Model Without Indicators

The first LSTM configuration used only raw OHLCV inputs. The model consisted of two stacked LSTM layers followed by dropout and dense layers. It was trained for 60

epochs and performed strongly in single-stock forecasting, showing stable behavior during training and smooth response curves.

Model Summary: LSTM Without Indicators

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 20, 128)	68,608
dropout (Dropout)	(None, 20, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Table 5.8: Stacked LSTM Model Architecture Summary

LSTM Model With Indicators

A second variation was trained using additional engineered features, aligning with the indicator set used in the GRU experiments. This configuration improved the model's ability to generalize across multiple stocks, particularly within NIFTY index datasets.

Feature	GRU Model	LSTM Model
Architecture	128→64 GRU	128→64 LSTM
Parameters	Fewer	Slightly more
Long-term memory	Moderate	Stronger
Training speed	Faster	Slower
Use case	Noisy short-term data	Trendy long-term sequences

Table 5.9: Comparison of GRU vs. LSTM Architectures

Model Summary: LSTM With Indicators

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 20, 128)	71,168
dropout_3 (Dropout)	(None, 20, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_4 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 32)	2,080
dense_5 (Dense)	(None, 1)	33

Table 5.10: Stacked LSTM Model Architecture Summary

A comparative observation between the two configurations showed that:

- The indicator-based LSTM model generalized better when trained on multiple companies.
- The non-indicator model remained highly effective for a single company, especially when trained on two-year datasets.
- This reinforces the idea that feature engineering provides benefit primarily when dataset diversity increases.

5.2.7 Performance Summary

The evaluation results demonstrated that the LSTM architecture provided a consistent balance between accuracy, generalization, and robustness. While GRU models trained faster and often matched performance on single-company prediction tasks, LSTM models achieved stronger results when scaling to larger datasets and incorporating additional features.

Overall, LSTM forms a key component of the modelling framework explored in this project and serves as a high-performing baseline against which more complex architectures such as CNN-LSTM and Transformer models were assessed.

5.3 Transformer Model

5.3.1 Introduction to the Transformer Architecture

The Transformer architecture was originally introduced for sequence-to-sequence learning tasks in natural language processing, where capturing complex contextual relationships and long-range dependencies is essential. Unlike traditional recurrent neural network architectures, Transformers do not rely on sequential processing. Instead, they use self-attention mechanisms that allow the model to evaluate the importance of different time steps relative to one another within the input sequence.

This mechanism enables the network to weigh which portions of the historical sequence are most relevant to the prediction task at any given time. Because attention is computed in parallel rather than iteratively, Transformers are computationally efficient at scale and capable of modeling longer temporal patterns than recurrent models without degradation in memory retention.

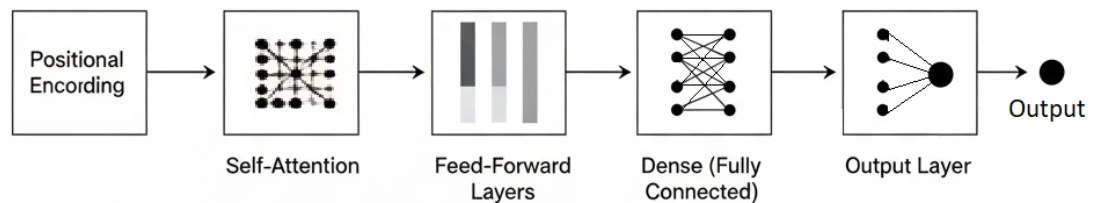


Figure 5.3: Architecture of the Transformer block.

5.3.2 Motivation for Using Transformers in Time-Series Forecasting

Transformers have recently gained adoption in financial forecasting because of their ability to:

- Capture long-term dependencies without recursive backpropagation
- Process sequences in parallel, improving learning efficiency
- Adapt dynamically to context, allowing the model to automatically prioritize relevant historical points

Stock movement patterns often depend on trends that evolve gradually, followed by sudden shifts triggered by market sentiment or volatility changes. The attention mechanism provides an analytical advantage because it allows the model to focus on these critical points rather than uniformly weighting past observations.

5.3.3 Mathematical Model for Transformer Encoder

For the Transformer encoder used in this project, the core computation is based on self-attention and position-wise feed-forward layers. Let $X \in \mathbb{R}^{T \times d_{\text{model}}}$ denote the input sequence of length T , after token/feature embedding and positional encoding.

Self-Attention Mechanism

$$\begin{aligned}
 Q &= XW_Q, \quad K = XW_K, \quad V = XW_V && \text{Linear Projections} \\
 A &= \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) && \text{Attention Weights} \\
 \text{SA}(X) &= AV && \text{Self-Attention Output}
 \end{aligned}$$

For multi-head self-attention with H heads:

$$\begin{aligned}
 \text{head}_h &= \text{SA}(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)}) \\
 \text{MHA}(X) &= \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O
 \end{aligned}$$

Transformer Encoder Layer Each encoder layer consists of a multi-head attention block and a position-wise feed-forward network (FFN), with residual connections and layer normalization:

$$\begin{aligned}
 H'_1 &= \text{LayerNorm}(X + \text{MHA}(X)) && \text{Residual + Norm after MHA} \\
 \text{FFN}(H'_1) &= \phi(H'_1W_1 + b_1)W_2 + b_2 && \text{Position-wise Feed-Forward} \\
 H_1 &= \text{LayerNorm}(H'_1 + \text{FFN}(H'_1)) && \text{Final Encoder Output for Layer}
 \end{aligned}$$

where $\phi(\cdot)$ is typically a ReLU or GELU activation.

Symbol	Description
X	Input sequence matrix of shape $T \times d_{\text{model}}$, where T is the sequence length (e.g., 20 days) and d_{model} is the embedding dimension. Includes feature embeddings and positional encodings.
Q, K, V	Query, Key, and Value matrices derived from X via learned linear projections. Used to compute attention scores and context.
Continued on next page	

Symbol	Description
W_Q, W_K, W_V	Projection weight matrices mapping the input X into query, key, and value spaces for single-head attention.
$W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$	Projection weight matrices for head h in multi-head attention. Each head learns its own subspace representation.
W_O	Output projection matrix applied to the concatenated multi-head outputs.
d_k	Dimensionality of the key (and query) vectors, used to scale the dot product in the attention mechanism.
A	Attention weight matrix obtained after applying the softmax function to the scaled dot products $QK^\top$. Each row represents a normalized attention distribution over time steps.
$\text{SA}(X)$	Single-head self-attention output, computed as AV , representing a weighted combination of value vectors.
head_h	Output of the h -th attention head. Each head focuses on different aspects or positions within the input sequence.
$\text{MHA}(X)$	Multi-head attention output formed by concatenating all heads and applying a final linear transformation.
H'_1	Intermediate hidden representation after adding the residual connection ($X + \text{MHA}(X)$) and applying layer normalization.
$\text{FFN}(\cdot)$	Position-wise feed-forward network applied independently to each time step, usually consisting of two linear layers with a nonlinearity.
W_1, W_2	Weight matrices of the feed-forward network, expanding and then projecting back the feature dimension.
b_1, b_2	Bias vectors associated with the feed-forward layers.
$\phi(\cdot)$	Nonlinear activation function applied in the FFN (e.g., ReLU or GELU).
H_1	Final output representation of the encoder layer after the second residual connection and layer normalization. This output is used either as input to the next encoder layer or for downstream prediction.
LayerNorm	Layer normalization function that normalizes activations across feature dimensions to stabilize and accelerate training.
Continued on next page	

Symbol	Description
Concat	Concatenation operation combining the outputs of multiple attention heads along the feature dimension.

Table 5.11: Key notations and equations for the Transformer encoder layer used in this project

5.3.4 Advantages and Disadvantages of Transformers

Table 5.12: Analysis of a Deep Learning Model (Likely Transformer/Self-Attention-based) for Time Series Prediction

Category	Observations
Advantages	• Able to learn long-range and non-linear dependencies • Scales efficiently to large datasets • Attention mechanism helps prioritize meaningful time steps • Performs well when incorporating multiple feature types
Disadvantages	• Requires substantially more data to train effectively • More computationally demanding during both training and tuning • Prone to underfitting on small datasets due to model complexity
Contextual Limitation	• Underperformed when trained only on NIFTY50 or NIFTY200 datasets • Required full multi-stock data to achieve stable prediction patterns • Less effective compared to LSTM when only OHLCV inputs were available

5.3.5 Role of Transformers in This Project

Transformers were included to examine how modern attention-based architectures perform relative to RNN-based approaches in forecasting next-day closing prices. The evaluation focused on determining whether the architecture could uncover higher-level temporal structure across multiple stocks in the Indian market, especially when technical indicators were included.

The experiments demonstrated that Transformers required significantly more data than recurrent architectures to obtain meaningful predictions. When trained using only raw OHLCV data or limited index-based datasets, the model typically underfit and produced weaker forecasting results. However, when trained on the full dataset containing multiple NSE-listed companies and additional features, the performance improved and moved closer to the accuracy produced by tuned LSTM models.

5.3.6 Trained Transformer Model Configurations Transformer Model Without Indicators

The first configuration used raw OHLCV features with a sequence length of 20 time steps. The model consisted of two stacked transformer encoder layers followed by a global pooling layer and fully connected output network. It was trained for 60 epochs using a batch size of 16.

Table 5.13: Model Architecture Summary

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 20, 5)	0
add (Add)	(None, 20, 5)	0
transformer_encoder (TransformerEncoder)	(None, 20, 5)	1,194
transformer_encoder_1 (TransformerEncoder)	(None, 20, 5)	1,194
global_average_pooling1d (GlobalAveragePooling1D)	(None, 5)	0
dropout_8 (Dropout)	(None, 5)	0
dense_6 (Dense)	(None, 64)	384
dropout_9 (Dropout)	(None, 64)	0
dense_7 (Dense)	(None, 1)	65

Transformer Model With Indicators

A second configuration incorporated the same set of engineered features used in previous GRU and LSTM experiments. This increased the model's input dimensionality and provided more structural information to the attention mechanism.

The model architecture remained similar, with stacked encoder blocks and dropout regularization, but required more parameters due to the increased feature size. It was also trained for 60 epochs using a batch size of 16.

5.3.7 Performance Summary

The performance evaluation showed that Transformers demonstrated potential when data volume and feature diversity increased. While the architecture did not outperform the LSTM model in this project, the experiments confirmed the following trends:

Table 5.14: Model Architecture Summary

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 20, 10)	0
add_1 (Add)	(None, 20, 10)	0
transformer_encoder_1 (TransformerEncoder)	(None, 20, 10)	3,124
transformer_encoder_2 (TransformerEncoder)	(None, 20, 10)	3,124
global_average_pooling1d (GlobalAveragePooling1D)	(None, 10)	0
dropout_13 (Dropout)	(None, 10)	0
dense_12 (Dense)	(None, 64)	704
dropout_14 (Dropout)	(None, 64)	0
dense_13 (Dense)	(None, 1)	65

Table 5.15: Comparison of Recurrent Neural Network (RNN) Architectures

Model	Strength	Weakness	When to Use
GRU	Light & fast	May miss long-term patterns	Quick prototypes
LSTM	Strong memory (handles vanishing gradient)	Slightly slower, more complex architecture	Stable trends
Transformer	Handles long-range relations, scalable, parallelizable	Heavier, needs more data	Advanced / larger datasets

- Transformers benefit substantially from broader training examples across multiple companies.
- Technical indicators improved stability and error consistency compared to using raw values alone.
- The architecture performed best on the largest dataset tested but remained close in accuracy to LSTM rather than exceeding it.

These outcomes suggest that the Transformer architecture is viable for stock forecasting when sufficient data is available and can serve as a strong model candidate as future iterations incorporate sentiment analysis, intraday signals, or larger multi-stock sequencing.

5.4 Conv1D + LSTM Hybrid Model

5.4.1 Introduction to the Hybrid Architecture

The Conv1D + LSTM hybrid model combines the strengths of convolutional neural networks (CNNs) and recurrent architectures to model temporal structure in financial time-series data. The 1-Dimensional Convolutional layers act as feature extractors that identify local temporal patterns, such as small waveforms, trend shifts, or repeating microstructures found in price movements. These extracted temporal feature maps are then passed to an LSTM layer, which processes them sequentially to learn longer-term dependencies and temporal relationships.

This layered design allows the model to benefit from both local pattern detection offered by CNNs and sequence-level learning achieved through recurrent memory. The architecture is well-suited for forecasting scenarios where short-term fluctuations accumulate into broader directional trends, a behavior frequently observed in equity price movement.

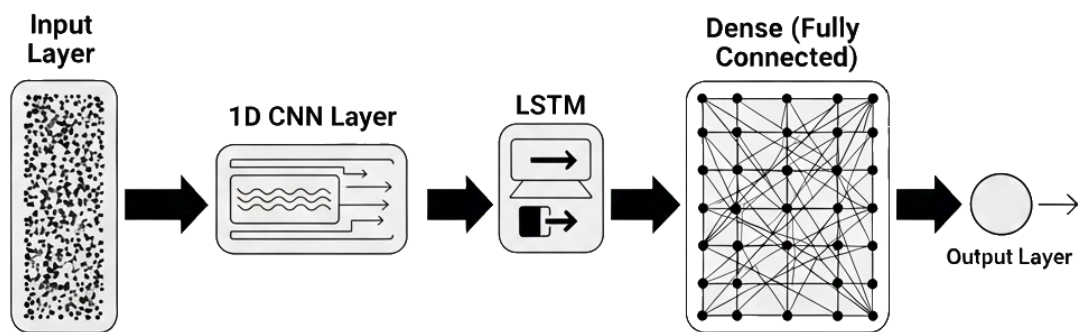


Figure 5.4: Architecture of the CNN + LSTM block.

5.4.2 Motivation for Using a Hybrid Model in Time-Series Forecasting

Pure sequential models such as LSTM or GRU tend to treat all time steps equally, which may dilute key short-term patterns—especially in financial data where recent movements and volatility surges may carry higher predictive weight. By contrast, convolutional layers selectively amplify short-term structures before passing them into the recurrent model.

This hybrid design helps address:

- Rapid fluctuation capture
- Local trend compression
- Feature smoothing before sequence modeling

As a result, Conv1D + LSTM models can learn meaningful representations from noisy datasets, which aligns well with the nature of stock market signals.

5.4.3 Advantages and Disadvantages of the Hybrid Model

Table 5.16: Analysis of a Hybrid Deep Learning Model (Likely CNN-LSTM)

Category	Observations
Advantages	• Captures both short-range (CNN) and long-range (LSTM) dependencies • More resilient to noise than LSTM-only models • Can extract meaningful patterns even with moderate feature engineering • Performs well when trained on large multi-stock datasets
Disadvantages	• Requires more data than simple recurrent models • Higher computational cost and slower convergence • Hyperparameter tuning for convolution and LSTM layers increases complexity
Contextual Limitations	• Underperformed when trained on only NIFTY50/NIFTY200 datasets due to limited sequence diversity • Performed best when trained using the full multi-company dataset

5.4.4 Role of the Hybrid Model in This Project

The hybrid Conv1D + LSTM model was included to evaluate whether multi-stage temporal feature extraction improves forecasting performance compared to pure recurrent or attention-based architectures. The model aligned well with the goal of understanding the impact of structured feature extraction on prediction accuracy.

The experiments showed that the hybrid architecture performed better when trained using multiple companies rather than single-company data. In single-stock configurations, the Conv1D layers often extracted redundant short-term behavior that the LSTM could already capture, providing little benefit. However, when the training dataset expanded to include a diverse range of stocks, the convolutional

filters identified transferable short-term market dynamics, improving generalization stability.

5.4.5 Trained Configurations of the Hybrid Model

The hybrid model followed the structure:

Figure 5.5: Conceptual Diagram of a CNN-LSTM Model Architecture



Hybrid Model Without Indicators

The first version of the hybrid model used only OHLCV features as input. It performed well when trained on larger datasets and produced smoother and more stable predictions compared to Transformer models trained on the same data volume. The absence of indicators forced the convolutional layers to detect raw micro-patterns in the closing sequence and local relationships in open-high-low volume dynamics.

Table 5.17: Model Architecture Summary

Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, 15, 5)	0
conv1d_2 (Conv1D)	(None, 15, 64)	1,024
conv1d_3 (Conv1D)	(None, 15, 64)	12,352
lstm_3 (LSTM)	(None, 64)	33,024
dense_10 (Dense)	(None, 32)	2,080
dropout_11 (Dropout)	(None, 32)	0
dense_11 (Dense)	(None, 1)	33

Hybrid Model With Indicators

A second version incorporated the engineered features used in earlier experiments. Including trend-based and volatility-based indicators expanded the dimensional richness of the input representation, providing additional structural cues for the convolutional filters. This configuration demonstrated improved generalization but required larger dataset scope to avoid overfitting.

The evaluation showed that among the tested indicator combinations, the feature set Close, MA5, and MA20 produced the most consistent results. Simpler trend-based indicators appeared to complement the convolutional layers more effectively than more volatile constructs such as ATR or RSI.

Table 5.18: Model Architecture Summary (Likely CNN-LSTM)

Layer (type)	Output Shape	Param #
input_layer_7 (InputLayer)	(None, 15, 10)	0
conv1d (Conv1D)	(None, 15, 64)	1,984
conv1d_1 (Conv1D)	(None, 15, 64)	12,352
lstm_2 (LSTM)	(None, 64)	33,024
dense_14 (Dense)	(None, 32)	2,080
dropout_15 (Dropout)	(None, 32)	0
dense_15 (Dense)	(None, 1)	33

5.4.6 Performance Summary

The Conv1D + LSTM hybrid model showed strong potential as a forecasting framework when sufficient training data was available. While the architecture did not surpass the LSTM model in single-stock configurations, it performed competitively when trained using large, multi-stock datasets and selected indicators. The results demonstrate that hybrid models can provide value in forecasting settings where both short-term fluctuations and broader momentum behaviors must be combined into a unified predictive structure.

Chapter 6

Results and Analysis

6.1 Overview

This chapter presents the results obtained from evaluating multiple machine learning architectures for forecasting next-day closing prices of Indian stock market equities. The models were assessed on both single-company datasets and broader market-level datasets to understand how input size, model complexity, and feature engineering influence predictive accuracy. Evaluation includes quantitative error metrics, visual prediction comparison, convergence behavior, and model generalization characteristics.

6.2 Training and Validation Behavior

Across experiments, all models showed decreasing loss curves during training, confirming the ability to learn meaningful temporal dependencies. However, their learning dynamics differed based on dataset scope and model capacity.

Even when just using the closing price overtime of the stocks shows great learning potential and thus confirms the ability of models to give insight on market movement.

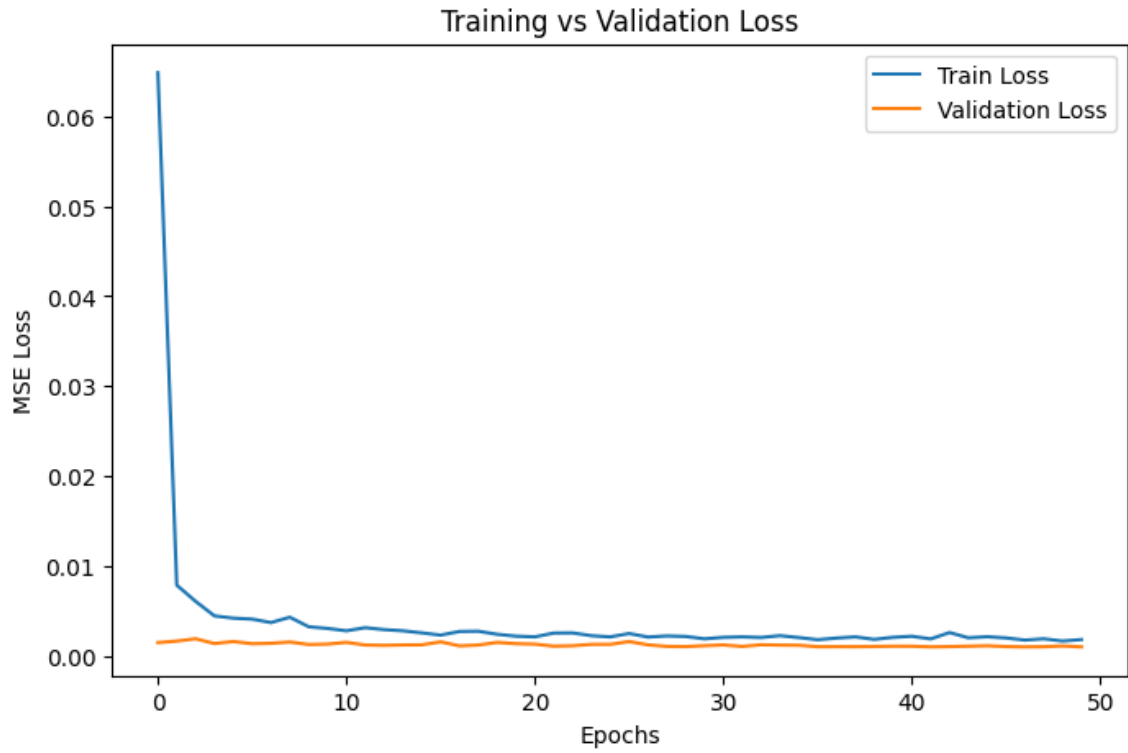


Figure 6.1: GRU without Indicators

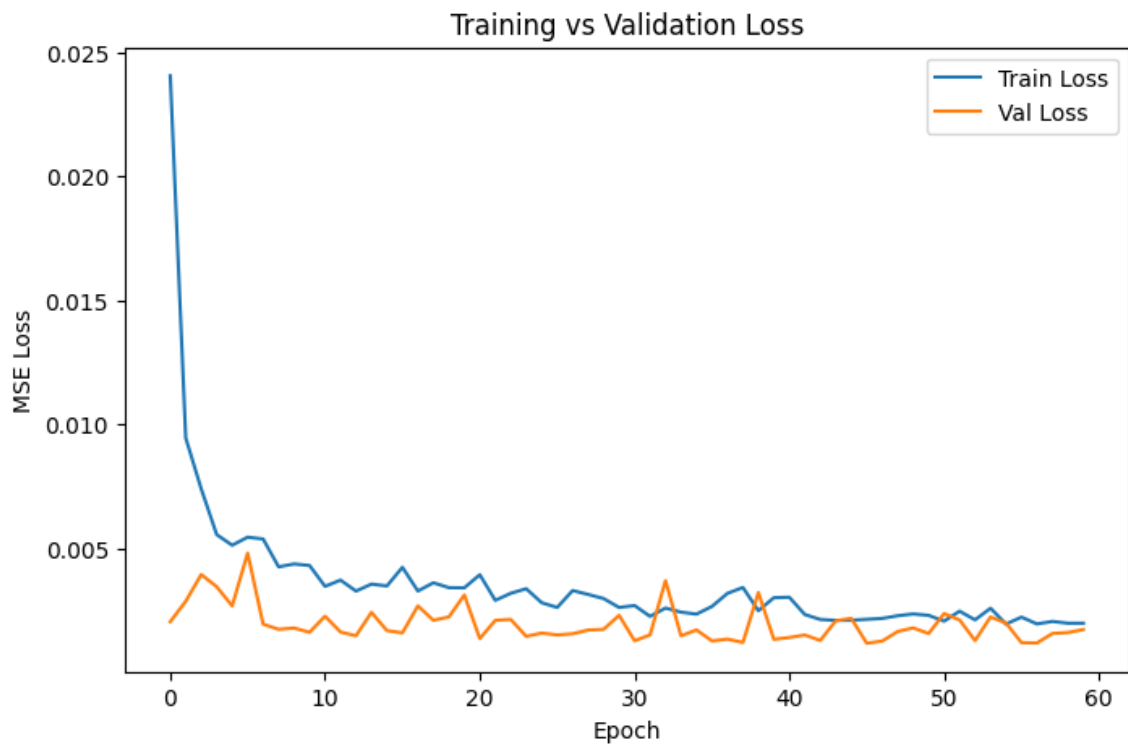


Figure 6.2: GRU with Indicators Validation Behaviour

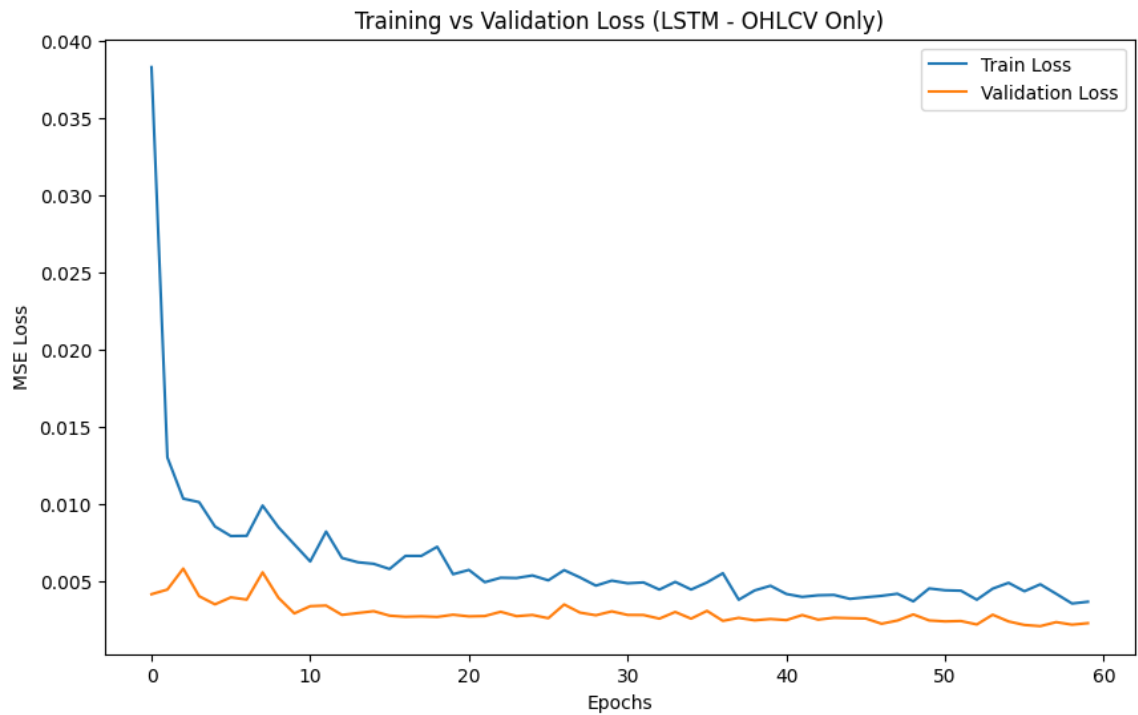


Figure 6.3: LSTM without Indicators Validation Behaviour

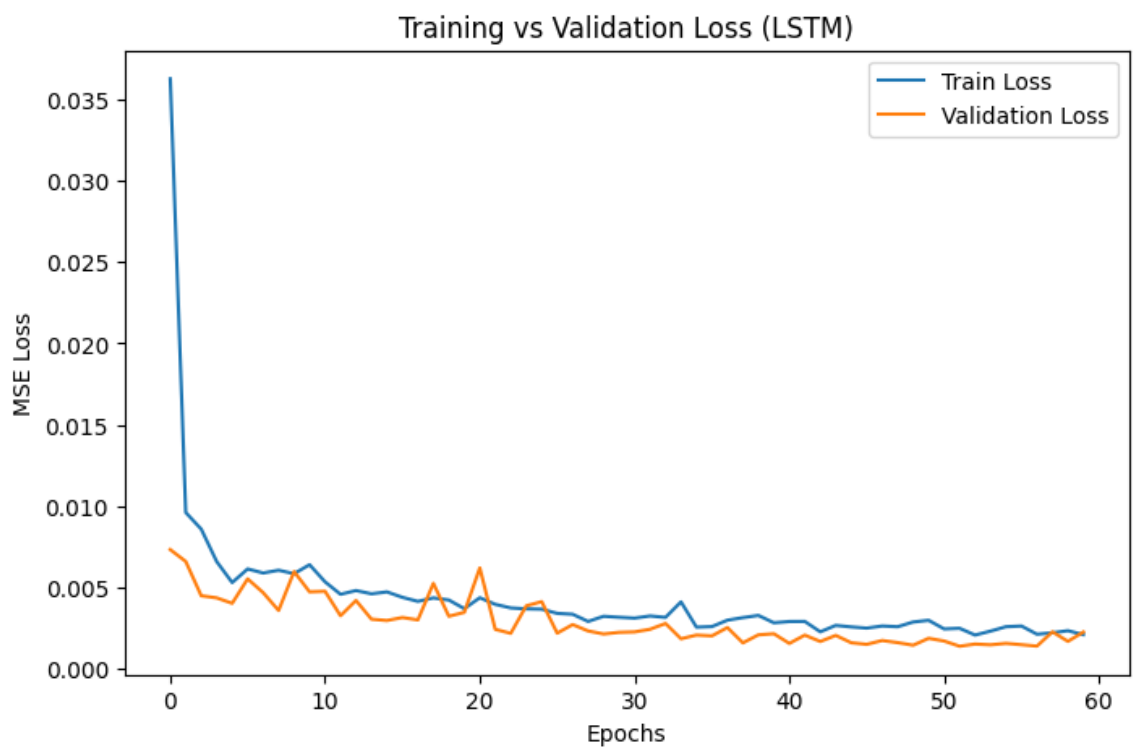


Figure 6.4: LSTM with Indicators Validation Behaviour

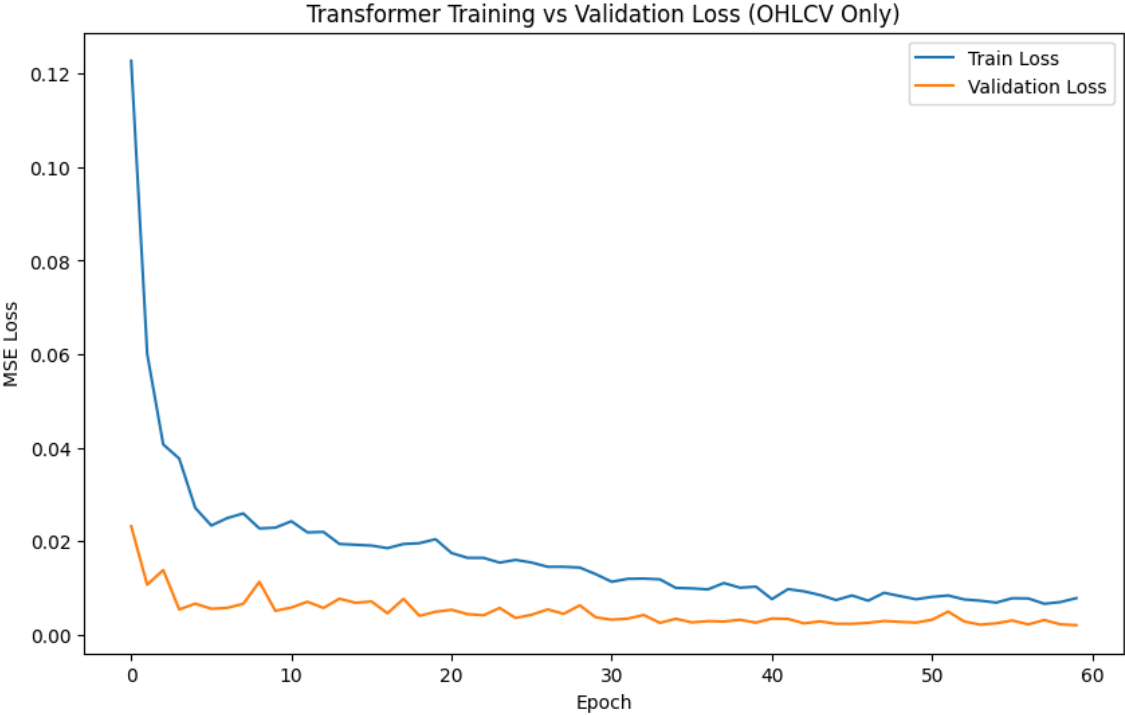


Figure 6.5: Transformers without Indicators Validation Behaviour

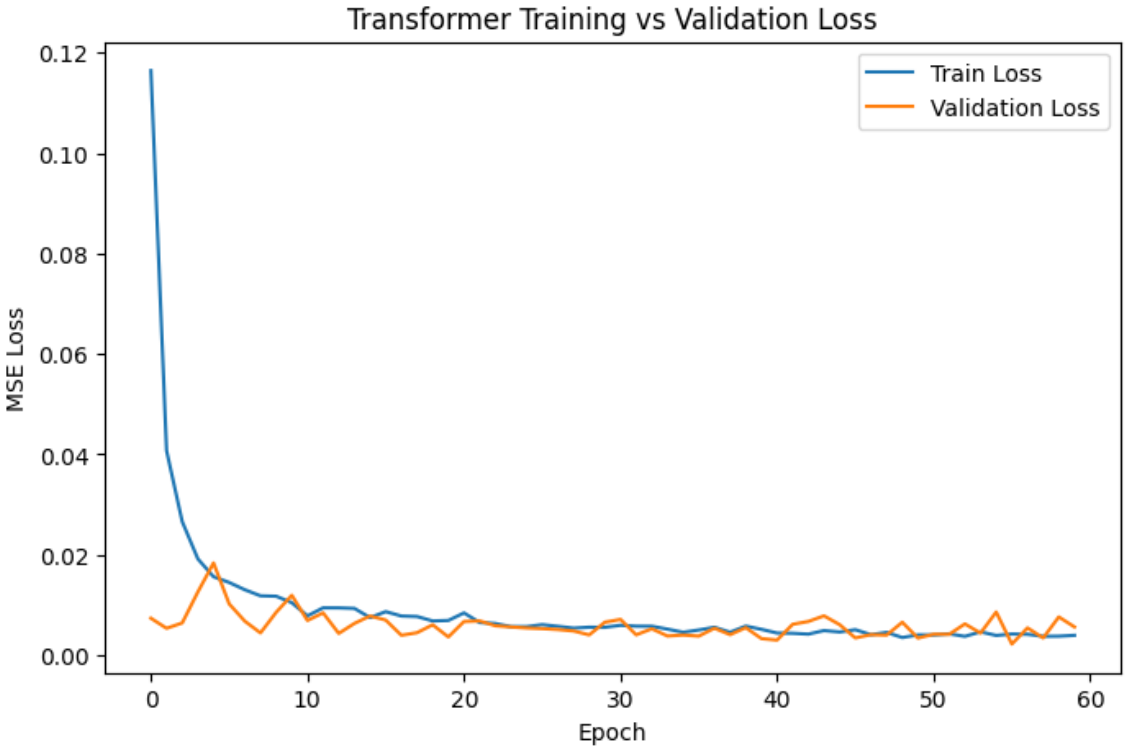


Figure 6.6: Transformers with Indicators Validation Behaviour

6.3 Single Company Data Used

Key findings from training Models with Single Company data and its analysis include:

- For single-company datasets, **GRU and LSTM models** converged fastest and most consistently, likely due to their lower parameter count relative to Transformers and CNN-LSTM networks.
- When only the **closing price** was used as the input feature, models fit extremely well and often achieved very low training loss, indicating possible **overfitting**.
- Larger architectures such as **Transformer and CNN-LSTM** demonstrated slower convergence and required significantly more data to avoid underfitting.
- Models like Transformers and CNN-LSTM when used for single company data showed significantly worse performance than simpler algorithm like GRU, LSTM and even classic algorithms like linear regression.
- This shows a significant disadvantage in using such algorithms there, when the data-set is smaller.

Following figures demonstrate the above findings

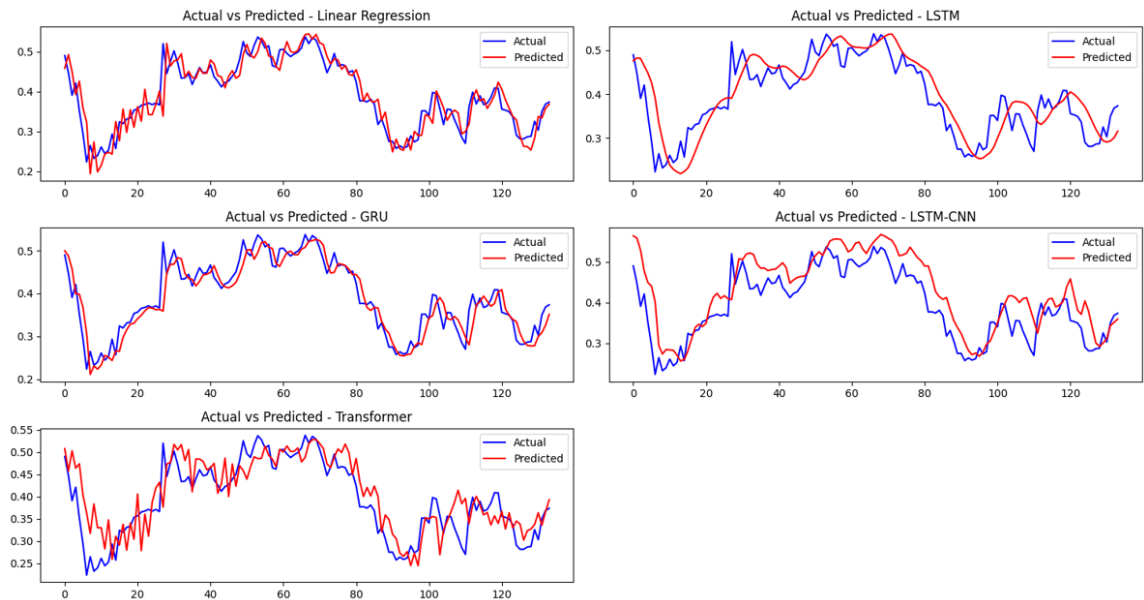


Figure 6.7: Regression Models on Single Company Data

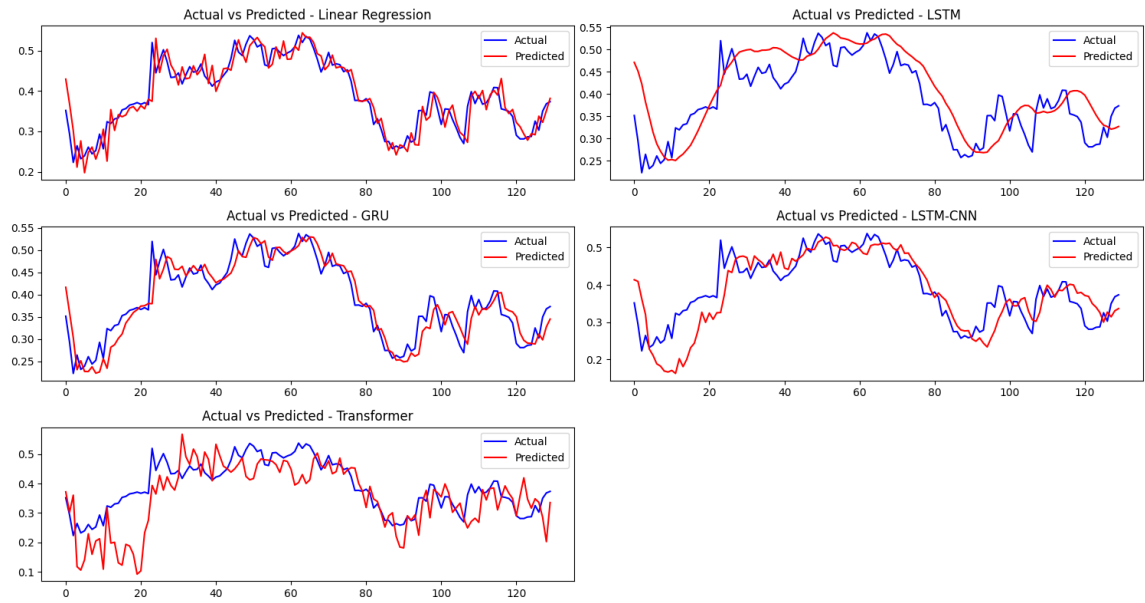


Figure 6.8: Regression Models on Single Company Data With Indicators



Figure 6.9: Regression Models Compared Without Indicators Included



Figure 6.10: Regression Models Compared with Indicators Included

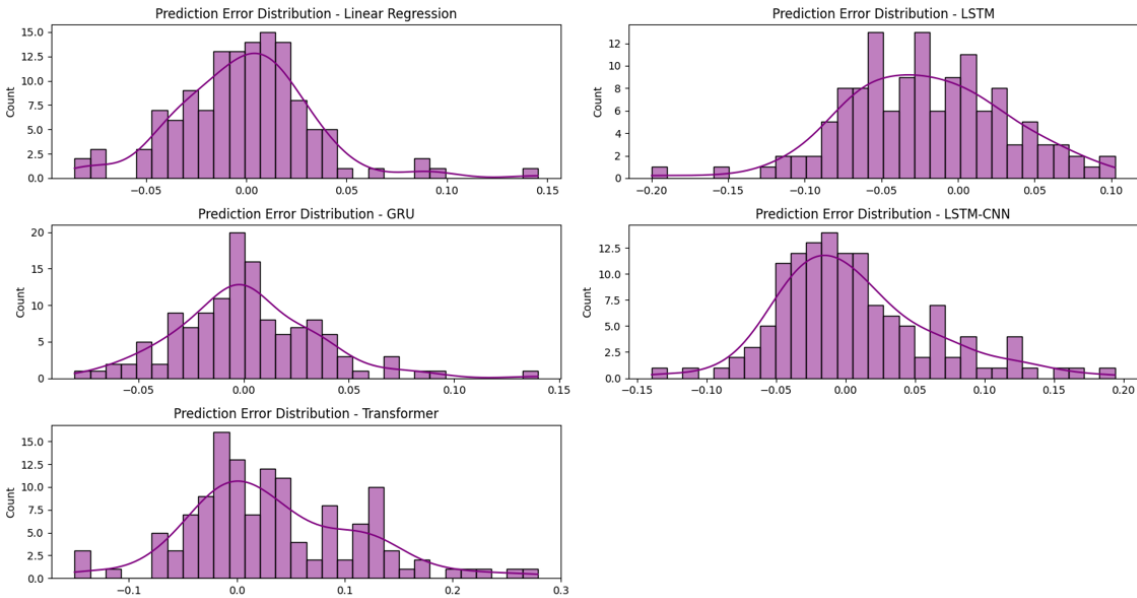


Figure 6.11: Prediction Errors for Regression Models on Single Company Data

6.3.1 Quantitative Evaluation Results

Table 6.1: Comparative Model Performance Across Datasets

Model Type	Single-Company Data	NIFTY50/200 Dataset	Full NSE Dataset
GRU	Best (fastest + accurate)	Poor generalization	Moderate
LSTM	Very good	Best generalization	Very good
CNN-LSTM	Underfits (small dataset)	Underfits (still limited)	Performs well, close to LSTM
Transformer	Underfits heavily	Weak performance	Performs well but not better than LSTM

The results clearly indicate that model complexity must match dataset size. Simpler recurrent models outperform complex architectures when data availability is limited.

6.4 Effect of Feature Engineering vs. Raw OHLCV Inputs

An important discovery in these experiments is that feature engineering did not always improve model performance.

Key conclusions:

- For single-company models, using only Close, Open, High, Low, Volume produced better results than using technical indicators, likely due to overfitting and noise sensitivity introduced by engineered features.
- For broader training datasets (NIFTY50, NIFTY200, or all NSE companies), adding indicators improved generalization and reduced error variance.

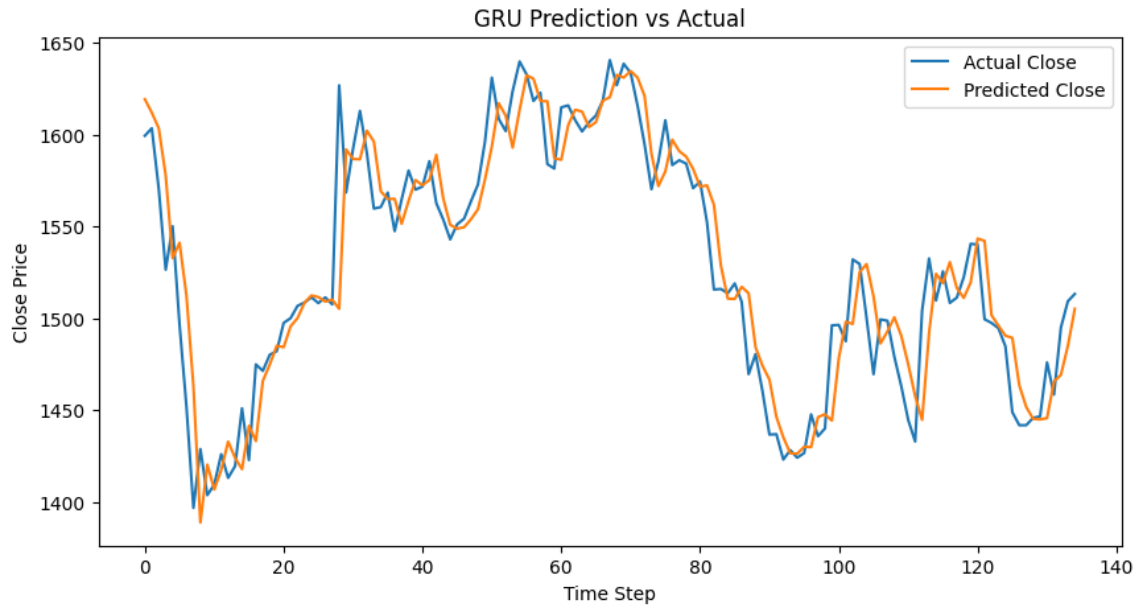


Figure 6.12: GRU with stock data

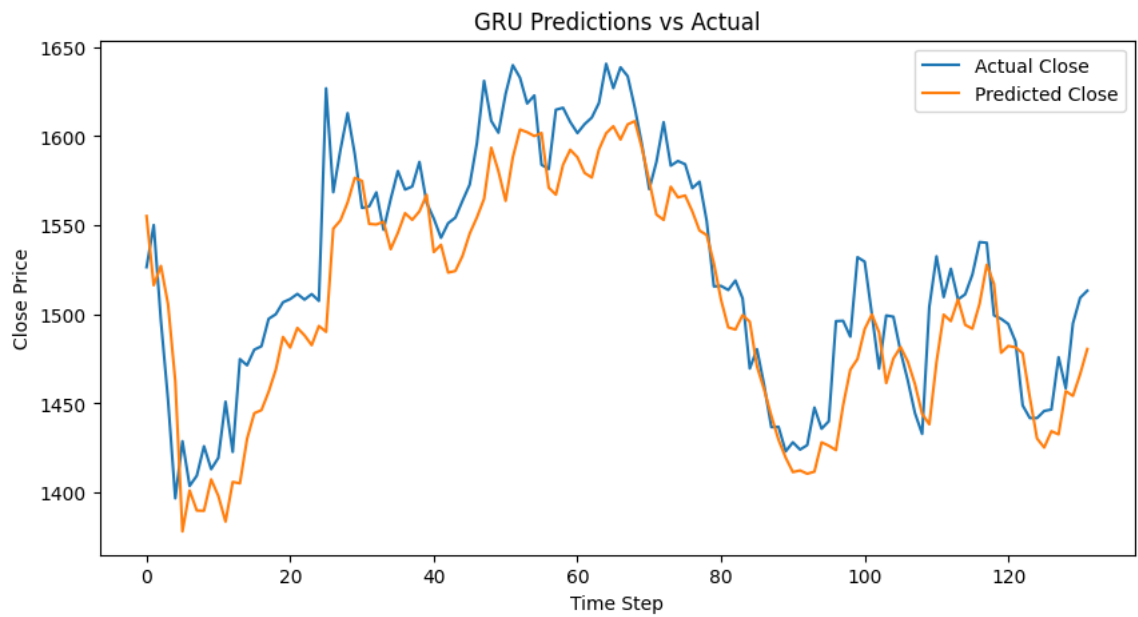


Figure 6.13: GRU with Indicators included (EMA (12, 26), RSI (14), ATR (14), OBV)

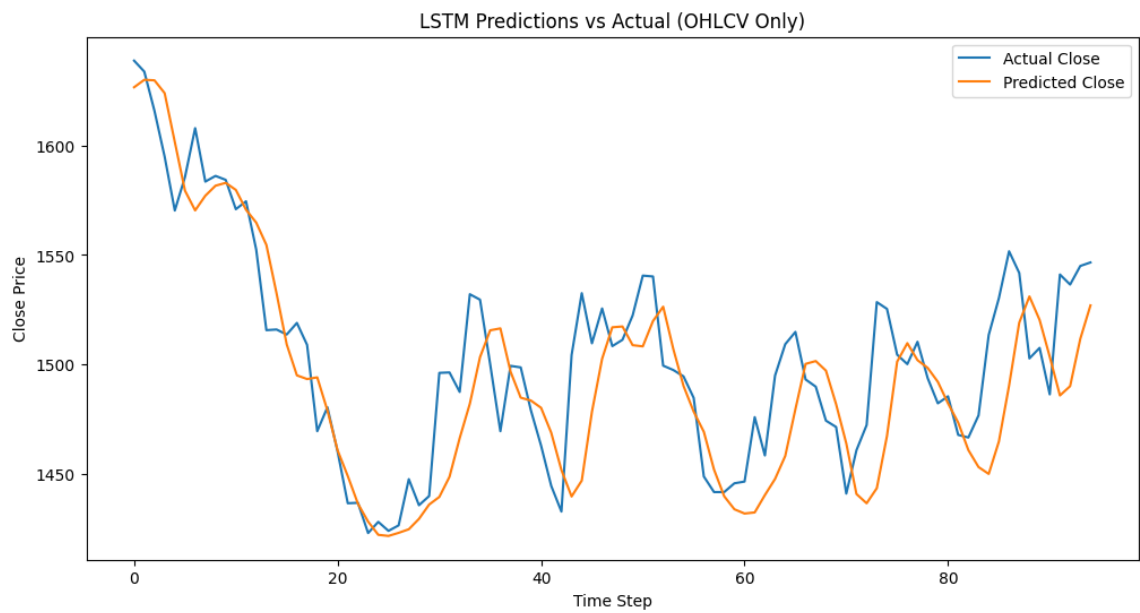


Figure 6.14: LSTM with stock data

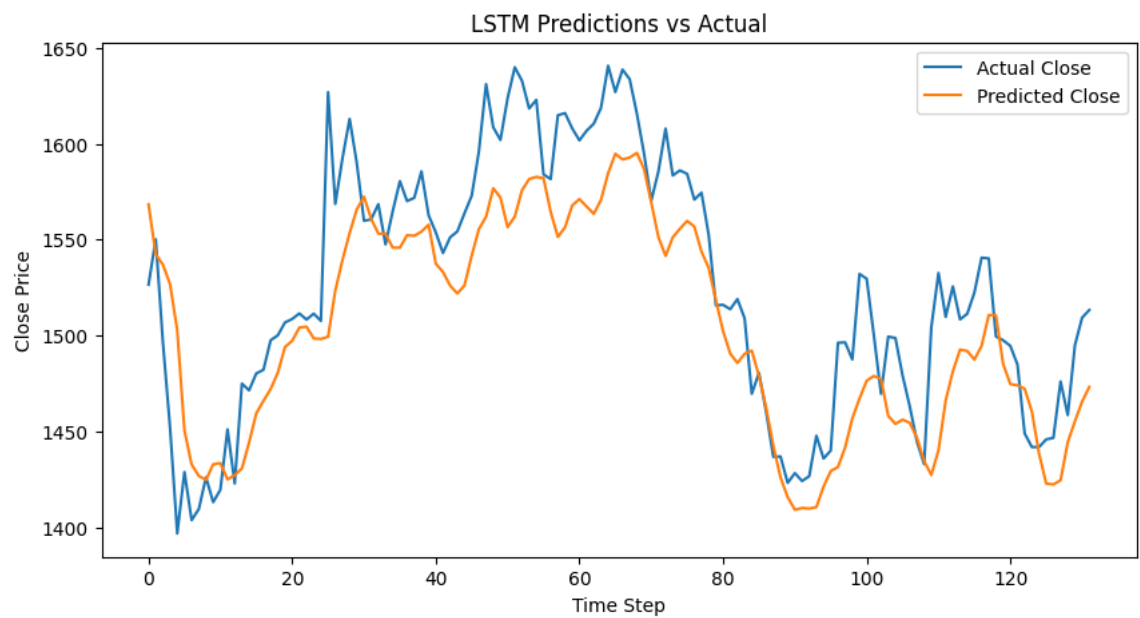


Figure 6.15: LSTM with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)

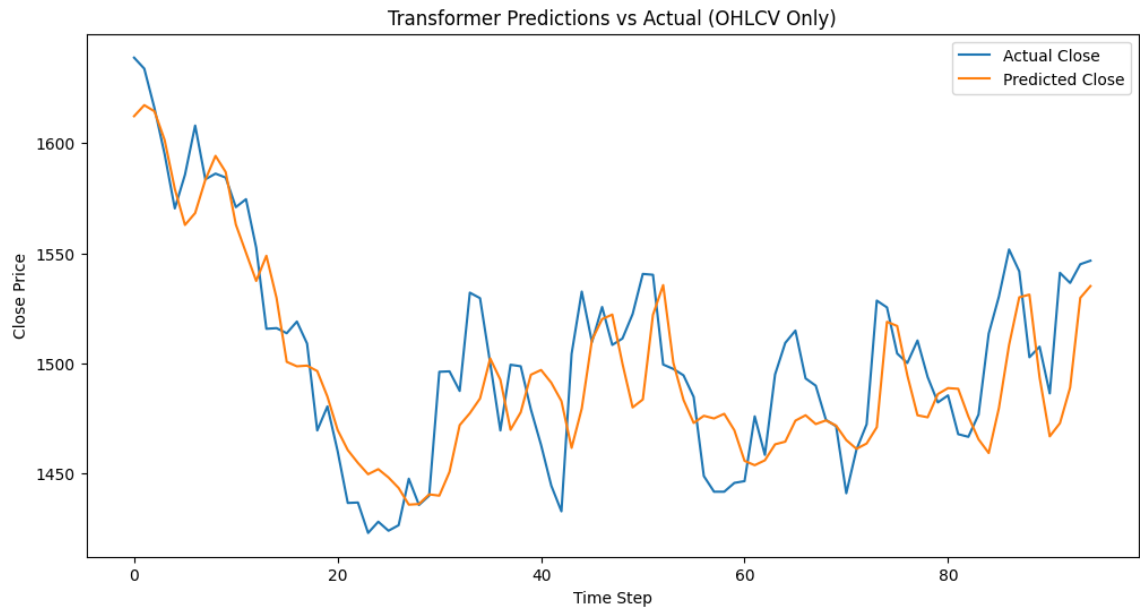


Figure 6.16: Transformers with stock data

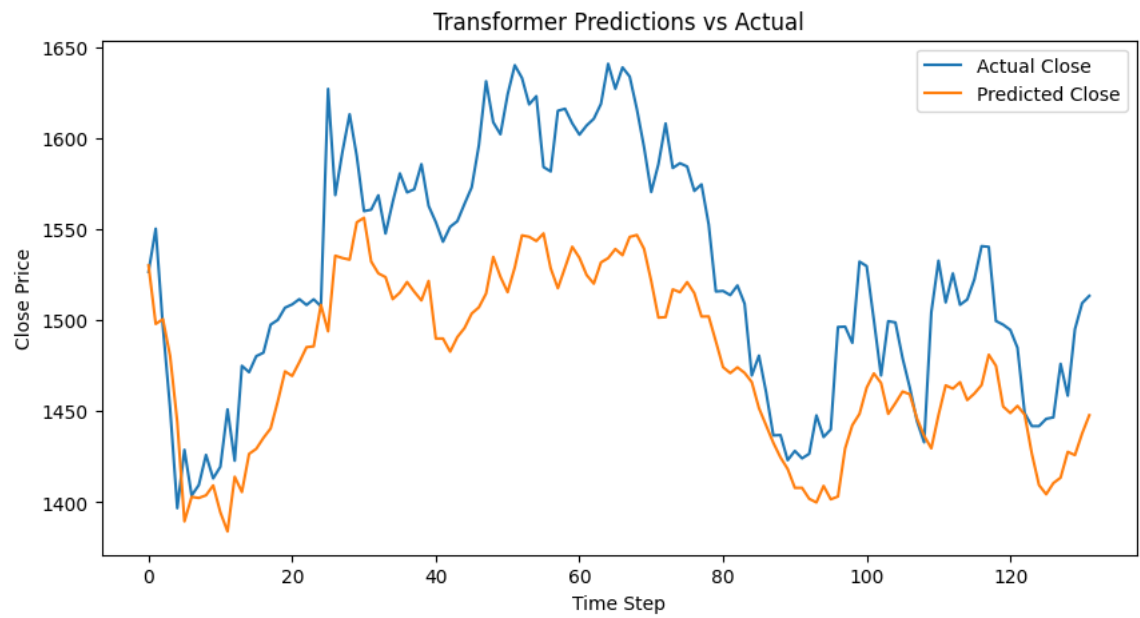


Figure 6.17: Transformers with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)

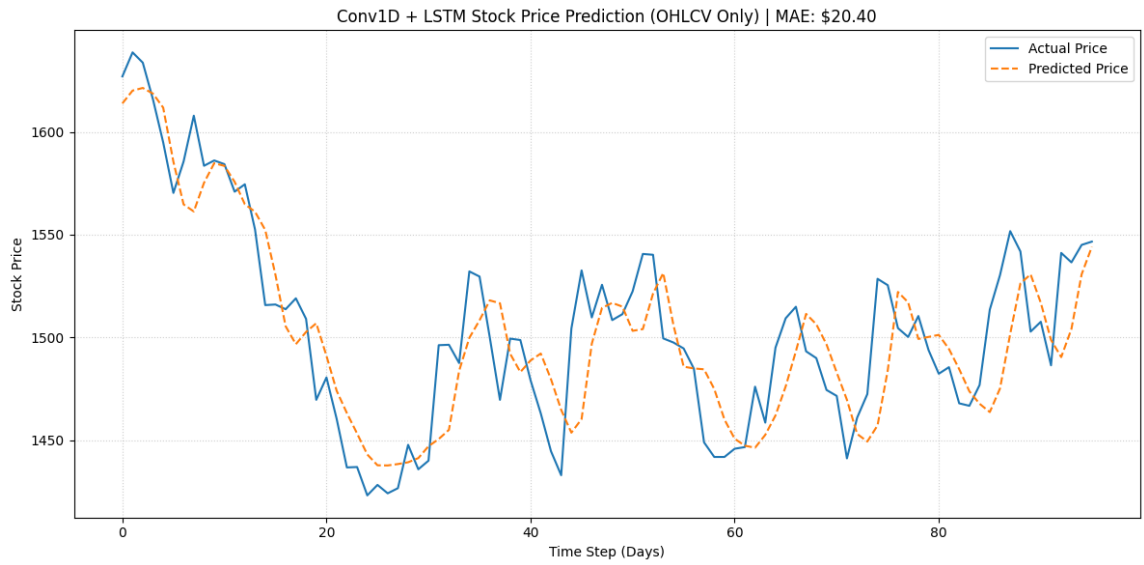


Figure 6.18: CNN-LSTM with stock data

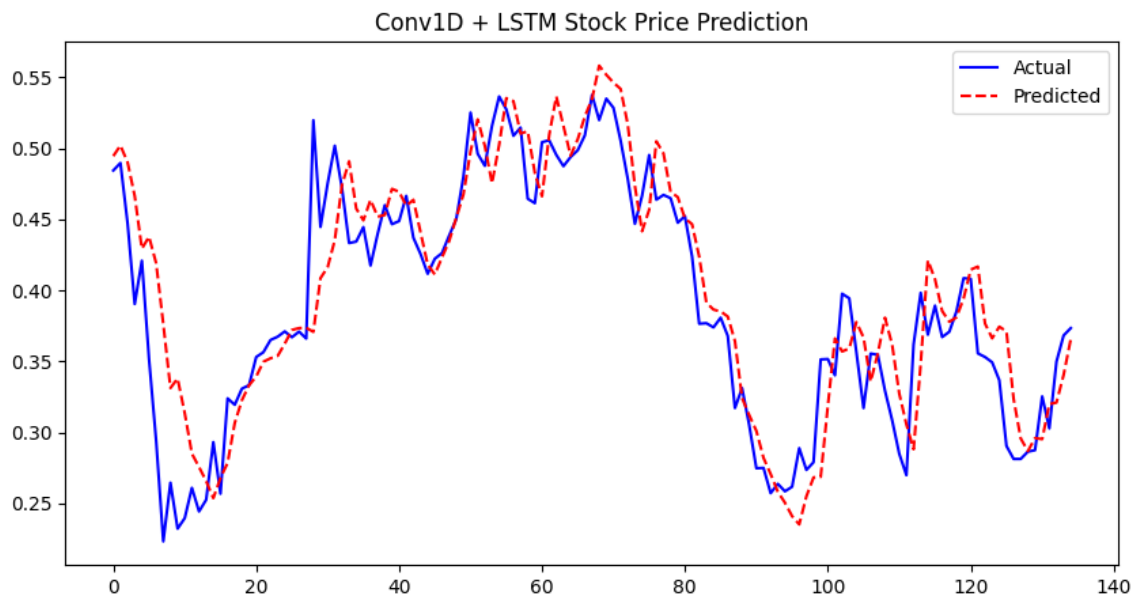


Figure 6.19: CNN-LSTM with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)

6.5 CNN-LSTM Feature Set Performance

For the CNN-LSTM architecture, three engineered feature combinations were tested:

Table 6.2: Feature Set Performance Comparison

Feature Set	Result
Close, MA5, MA20	Best performance
Close, EMA10, Standard Deviation	Moderate
Close return + STD20	Least consistent

This indicates that simple, trend-focused moving averages outperform more volatile or mathematically dense indicators in deep hybrid models.

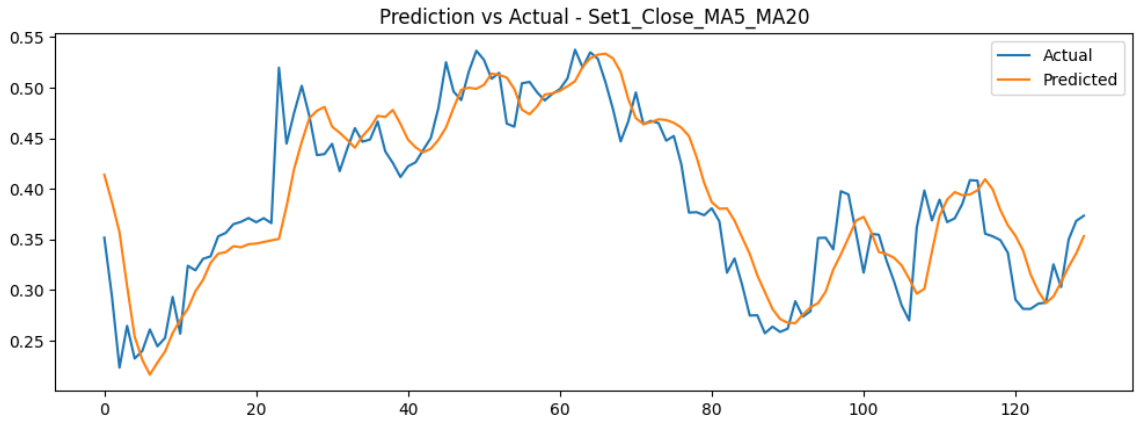


Figure 6.20: CNN-LSTM with Indicators (Close, MA5, MA20)

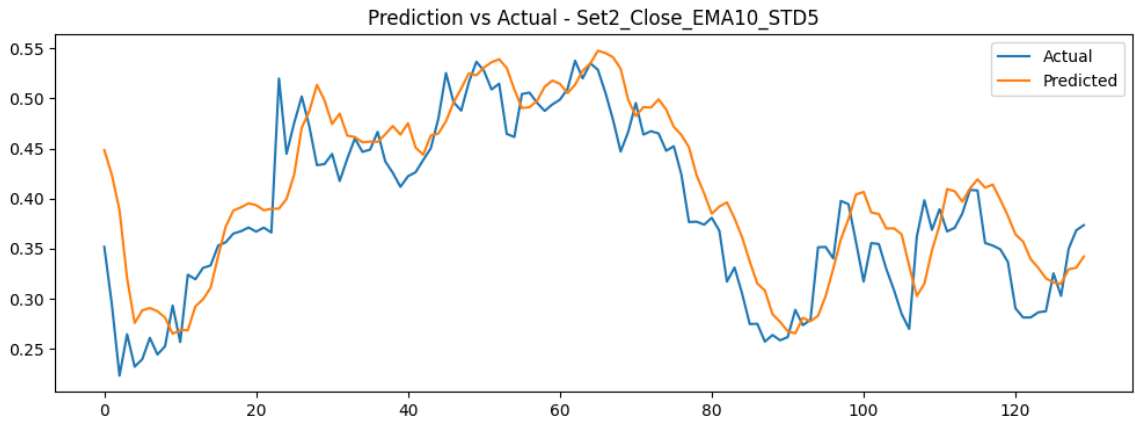


Figure 6.21: CNN-LSTM with Indicators (Close, EMA10, STD5)

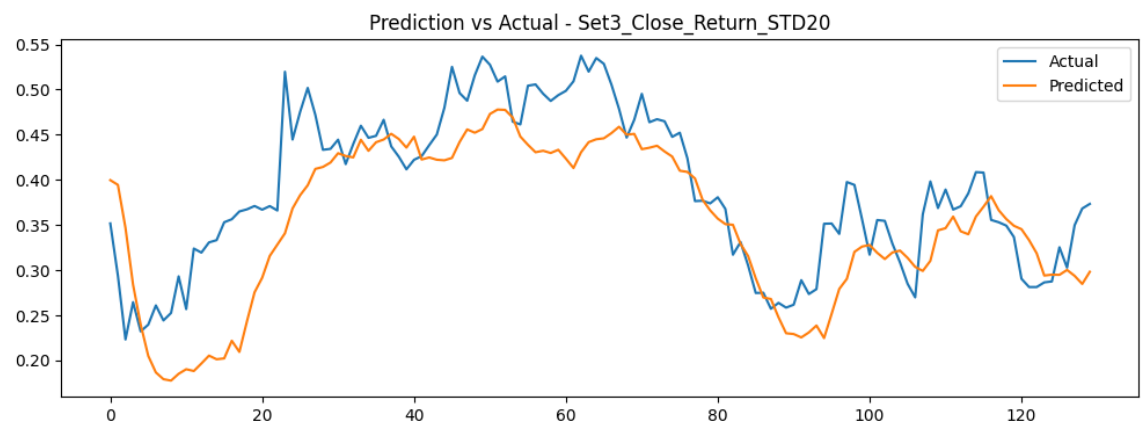


Figure 6.22: CNN-LSTM with Indicators (Close, Return, STD20)

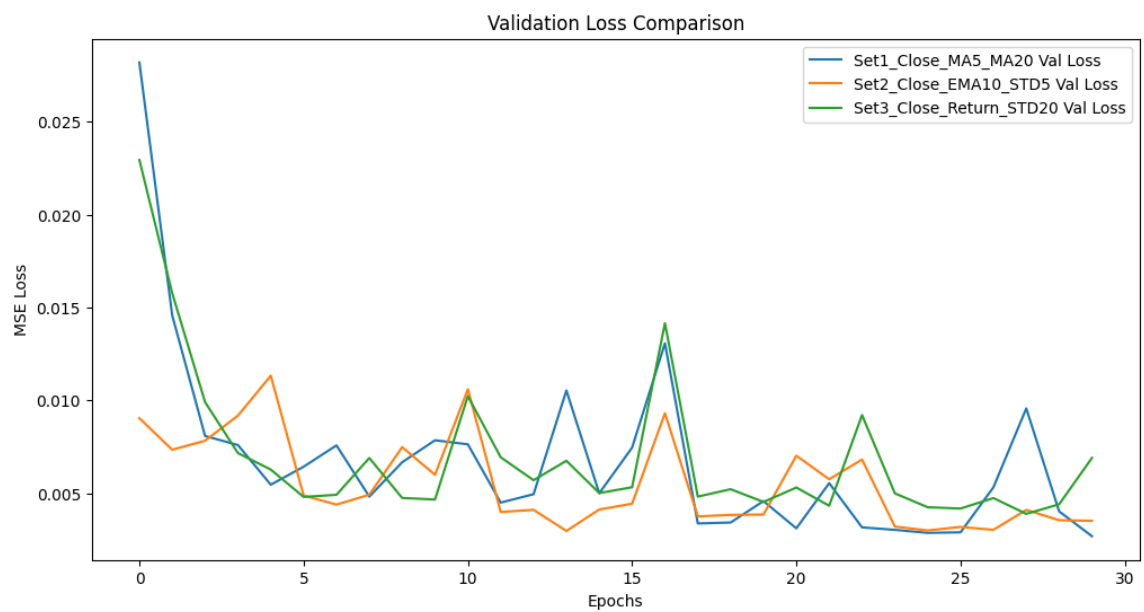


Figure 6.23: CNN-LSTM with Indicators Validation Loss Comparisons

6.5.1 Indicators tested and their purpose

Table 6.3: Strengths and Measures of Technical Indicator Sets

Indicator Set	What it Measures	Strengths
1. Close, MA5, MA20	Price trend over short (5-day) and medium (20-day) periods	Simple, easy to interpret; captures short- and medium-term momentum; identifies trend changes (crossovers)
2. Close, EMA10, STD	EMA10: weighted short-term trend; STD: price volatility	EMA reacts faster than simple MA; STD captures volatility (risk)
3. Close Return, STD20	Return captures percentage change , STD20 measures volatility of returns	Focuses on percentage movement rather than absolute price; STD20 captures medium-term volatility trends

Table 6.4: Summary of Technical Indicator Sets

Indicator Set	What it Measures	Usefulness in Stock Modeling
1. Close, MA5, MA20	Price trend over short (5-day) and medium (20-day) periods	Helps capture trend direction and short-term momentum , useful for regression or classification of up/down movement.
2. Close, EMA10, STD	EMA10: weighted short-term trend; STD: price volatility .	Useful to detect price momentum + risk ; volatility helps the model understand uncertainty and potential extreme moves.
3. Close Return, STD20	Return captures percentage change ; STD20 measures volatility of returns .	Good for risk-sensitive models and detecting high-volatility periods , can improve direction prediction.

6.6 Generalization vs. Specialization Behavior

Another major insight from these experiments is that:

- Models trained on a single company become highly specialized, fitting well to patterns unique to that stock but failing to generalize across companies.
- Models trained on larger aggregated datasets show better generalization, but require feature engineering and larger architectures to extract meaningful predictive structure.

This reflects a trade-off:

- Single-company models = accuracy in isolation
- Multi-stock models = robustness and scalability

6.7 Deployment and Practical Evaluation

When deployed into live prediction with recent 3-month price windows:

- LSTM and GRU showed stable point-wise forecasts suitable for short-term use.
- Transformer and CNN-LSTM predictions improved only when trained on very large multi-company datasets.
- Simpler models responded more smoothly, while complex architectures reacted more sharply to volatility.

Chapter 7

Summary and Conclusions

7.1 Summary of Work and Key Contributions

The primary objective of this research was to build an automated system capable of predicting the next-day closing stock price of Indian market equities using machine learning techniques informed by technical analysis. The project successfully developed and evaluated multiple models including GRU, LSTM, CNN-LSTM, and Transformer architecture using real historical data retrieved through the Kite Connect API.

In its final form, the project demonstrates that meaningful predictive performance can be achieved using deep learning-based time series modeling and that model behavior varies significantly depending on dataset size, architecture, and feature design. A complete working prototype system was also built, integrating live data retrieval, preprocessing, inference, and visual chart-based validation.

7.2 Problem Statement and Solution

Stock market forecasting is challenging due to volatility, noise, and nonlinear dependencies that traditional statistical and manual technical analysis struggle to model. Prior research often lacked experimentation across dataset scales, did not evaluate generalization capability, or relied only on accuracy without deployment validation.

This project addressed these gaps by:

- Comparing multiple model families rather than a single architecture
- Evaluating performance across single-company training, index-based datasets, and full-market datasets

- Testing models both with raw OHLCV inputs and with engineered technical indicator feature sets
- Implementing a live deployment and visualization pipeline for real-time inference

This systematic comparison framework provided insight into which models are best suited for various dataset conditions and use cases.

7.3 Major Findings and Significance

The experiments revealed several meaningful insights:

- GRU models performed best when trained on data from a single company, offering fast training and strong prediction accuracy even without engineered indicators.
- LSTM models were the most stable and generalizable architecture, performing well both on individual stock datasets and on NIFTY50/NIFTY200 multi-stock sets.
- Feature engineering did not universally improve results. For small datasets, raw OHLCV inputs often outperformed engineered features, likely due to reduced noise and decreased overfitting risk.
- Complex architectures like Transformers and CNN-LSTM models required large datasets to perform competitively. When trained only on index-level datasets, they underperformed simpler RNN models. However, when trained using all available NSE data, their performance improved significantly, approaching that of LSTM models.
- The best-performing feature combination for CNN-LSTM was Close, MA5, and MA20, suggesting that simple trend-focused transformations are more effective than volatile or mathematically dense indicators.

The significance of these findings lies in demonstrating that deep learning approaches remain highly promising for stock forecasting but must be matched appropriately to dataset scale and complexity. This work provides a practical roadmap for model selection and tuning in real-world financial prediction systems.

7.4 Limitations of the Study

While the project achieved its intended goals, several limitations remain:

- The dataset consisted only of daily closing data, excluding intra-day signals or volume profile dynamics.
- Technical indicators used were selected manually and were not optimized using automated feature selection or genetic search.
- The model evaluation period was limited, and live deployment testing did not incorporate transaction costs, execution slippage, or trading constraints.
- Only price-based data was used; no sentiment, macroeconomic, or alternative data sources were included.
- The real-time evaluation was observational and not connected to an automated trading engine.

These limitations do not diminish the results but highlight the complexity of building a fully production-ready forecasting system.

7.5 Future Work

Several opportunities exist to extend this research:

- Incorporate more diverse data sources, including sentiment data, financial news feeds, and macroeconomic indicators.
- Improve feature engineering, potentially using automated selection methods such as SHAP, PCA, or feature importance ranking.
- Expand model scope to intraday prediction and real-time streaming data.
- Refine hybrid architectures, including improved Transformer variants (e.g., Informer, FEDformer, or Time-Series Transformers).
- Develop a full backtesting and trading engine with portfolio simulation metrics such as maximum drawdown, Sharpe ratio, and profit factor.
- Extend generalization evaluation across sectors, volatility regimes, and economic conditions.

These improvements would help transition the model from a research prototype to a sophisticated decision support or automated execution system.

7.6 Conclusion

In conclusion, the research demonstrates that machine learning-driven technical analysis can offer valuable improvements in short-term stock price forecasting and lays the groundwork for more advanced predictive and automated trading systems. The insights gained regarding model performance, data scale sensitivity, and feature behavior contribute meaningfully to ongoing work in financial machine learning and provide a strong foundation for future advancements in intelligent stock market modelling.

References

- [1] W. Brock, J. Lakonishok, and B. LeBaron, “Simple technical trading rules and the stochastic properties of stock returns,” *The Journal of Finance*, 1992.
- [2] A. W. Lo, “The adaptive markets hypothesis,” *The Journal of Portfolio Management*, 2004.
- [3] E. F. Fama, “Efficient capital markets: A review of theory and empirical work,” *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970.
- [4] J. N. Tsitsiklis and D. P. Bertsekas, *Introduction to Probability*. Athena Scientific, 2002, Referenced widely for stochastic modelling foundations in finance.
- [5] G. Box and G. Jenkins, *Time Series Analysis: Forecasting and Control*. 1970.
- [6] R. S. Tsay, *Analysis of Financial Time Series*, 3rd Edition. John Wiley & Sons, 2010.
- [7] R. Engle, “Autoregressive conditional heteroscedasticity with estimates of uk inflation,” *Econometrica*, 1982.
- [8] C. Chatfield, *The Analysis of Time Series: An Introduction*, 4th Edition. Chapman and Hall, 1989.
- [9] J. J. Murphy, *Technical Analysis of the Financial Markets*. New York Institute of Finance, 1999.
- [10] S. B. Achelis, *Technical Analysis from A to Z*. 2001.
- [11] C. Neely, D. Rapach, J. Tu, and G. Zhou, “Forecasting the equity risk premium: The role of technical indicators,” *Management Science*, 2014.
- [12] C.-H. Park and S. Irwin, “What do we know about the profitability of technical analysis?” *Journal of Economic Surveys*, 2007.
- [13] F. Huang, S. Huang, and A. Huang, “Predictive modeling of the global stock markets using machine learning,” *Journal of Financial Risk Management*, 2022.
- [14] Y. Smarte and G. D. Ho, “Review of machine learning for stock prediction based on technical analysis,” *IEEE Transactions on ...*, 2021.

- [15] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [16] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, “Learning phrase representations using rnn encoder–decoder for statistical machine translation,” in *Proceedings of EMNLP*, 2014.
- [17] H. Zhu, *Stock prediction using artificial intelligence technology: A review*, ResearchGate, 2025.
- [18] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, 1998.
- [19] F. Jin, “Cnn prediction of the stock market indicators using a hybrid deep learning system,” *IEEE Transactions on ...*, 2024.
- [20] A. Vaswani et al., “Attention is all you need,” in *Proceedings of NIPS*, 2017.
- [21] F. Khanpuri, S. Khanpuri, and A. Khanpuri, “Sentiment analysis on fundamental data to predict stock prices,” *EAI Endorsed Transactions on AI and Robotics*, 2024.
- [22] F. Huang, S. Huang, and A. Huang, “Deep learning to llms: A survey on quantitative investment,” *IEEE Transactions on ...*, 2025.
- [23] F. Cao, S. Cao, and A. Cao, “Applying llms for stock forecasting,” *Proceedings of ICAIF '24*, 2024.
- [24] F. Cilingiroglu, *How sentiment is changing the stock price prediction market*, Forbes, 2025.
- [25] A. Gupta, “The role of ai in the stock market to enhance predictability & strategy,” *IJEER*, 2024.
- [26] F. Mokhtari, S. Mokhtari, and A. Mokhtari, *The impact of artificial intelligence in enhancing predictive stock trading*, Munich Personal RePEc Archive, 2021.
- [27] J. Lu, “The financial architecture of blockchains and open services,” *IRIS*, 2024.
- [28] F. Balasubramanian, S. Balasubramanian, and A. Balasubramanian, “Stock market analysis using biometric indicators,” *SHS Web of Conferences*, 2024.